

JÖNKÖPING UNIVERSITY
School of Engineering

Master Thesis

Automated Controller Tuning for a Two-Wheeled Self-Balancing Robot

A Physical Trial-Based Comparison of Cascade-PID and State-Feedback Parametrizations Using Differential Evolution

PAPER WITHIN *Industrial Product Realisation*

AUTHOR: *Denis Fetah & Edgardo Mejia*

Jönköping University
School of Engineering
Department of Product Development ▪ May 31st 2026

This final project work has been carried out at the School of Engineering, Jönköping University, in the subject area Industrial Product Realization. The authors take full responsibility for opinions, conclusions and findings presented.

Examiner:	Marco Ferreira Rodrigues Nogueira Santos
Supervisor:	Mirza Cenanovic, Jönköping University
Scope:	30 credits

Abstract

Two-wheeled self-balancing robots are inherently unstable systems that require carefully tuned controllers to maintain balance. Most traditional tuning methods are often manual, time consuming, and sensitive to changing system conditions. This thesis investigates how Differential Evolution (DE), a population-based optimization algorithm, can reliably tune controllers for a two-wheeled self-balancing robot directly through physical trials. The study also compares the performance and convergence behavior of two control structures: cascade-Proportional Integral Derivate (PID) control and Linear Quadratic Regulator (LQR)-form state-feedback control.

A quantitative and experimental methodology was used. Both controllers were implemented on the same robot platform and optimized using an identical DE-based workflow. The process for the optimization was divided into two phases consisting of global exploration followed by local refinement. Special focus was placed on objective function design to ensure that the optimizer rewarded genuine balancing behavior rather than unintended solutions. Several objective function iterations were tested and refined during physical experiments. System identification experiments were also conducted to evaluate the feasibility of analytical LQR implementation on the available hardware.

The results show that DE can successfully tune both cascade-PID and LQR-form controllers through physical trials when the optimization workflow and objective function are carefully designed. It was identified that objective functions based solely on oscillation amplitude were vulnerable to exploitative behavior such as wall leaning and drifting. A final objective function formulation combining pitch stability, lean penalties, velocity penalties, and runaway detection produced reliable optimization behavior. The cascade-PID controller achieved a low fall rate and slightly lower steady state velocity, while the LQR-form controller showed faster convergence and slightly better balancing performance overall. The study also found that direct analytical derivation of LQR gains was not feasible due to the inability to measure critical system parameters on the available hardware.

This thesis contributes a practical workflow for physical-trial controller optimization using Differential Evolution and demonstrates how objective function design strongly influences optimization success in unstable robotic systems. The findings provide guidance for future research and practical implementation of automated controller tuning for self-balancing robots.

Keywords: two-wheeled self-balancing robot, differential evolution, PID control, LQR, controller tuning, physical trial optimization, inverted pendulum

Acknowledgements

We would like to express our deepest gratitude to our supervisor Mirza Cenanovic for his guidance, feedback, and support throughout the duration of this project. His input was invaluable in shaping the direction of the research and improving the quality of the work. We would also like to thank the School of Engineering at Jönköping University for providing access to the hardware and laboratory facilities that made this work possible. Finally, we would like to thank our families and friends for their patience and encouragement throughout the process.

Sincerely,



Denis Fetah



Edgardo Mejia

Table of contents

1. Introduction **1**
- 1.1. Background . . . 1
- 1.2. Research Gap . . . 2
- 1.3. Problem Description . . . 3
- 1.4. Purpose and research questions . . . 3
- 1.5. Delimitations . . . 3
- 1.6. Outline of the thesis . . . 4

2. Theoretical framework **7**
- 2.1. Two Wheeled Robots (TWR) . . . 7
- 2.2. Inverted Pendulum . . . 7
  - 2.2.1. Lagrange Method . . . 9
- 2.3. Proportional, Integral, Derivate (PID) . . . 12
  - 2.3.1. Limitations of PID tuning . . . 13
- 2.4. Linear Quadratic Regulator (LQR) . . . 13
- 2.5. Optimization . . . 15
  - 2.5.1. Classical Optimization Methods . . . 15
  - 2.5.2. Metaheuristic Optimization . . . 15
- 2.6. Differential Evolution . . . 16
  - 2.6.1. Properties Relevant to Engineering Optimization . . . 16
  - 2.6.2. Applications in Control System Tuning . . . 17
- 2.7. Hardware and Implementation . . . 17
- 2.8. Sensors . . . 17
- 2.9. Motors . . . 19
  - 2.9.1. Motor Driver . . . 20
- 2.10. Microcontroller . . . 21
  - 2.10.1. SimpleFOC . . . 22

3. Methodology **23**
- 3.1. Research Design . . . 23
- 3.2. Research Process . . . 24
- 3.3. Control Architectures . . . 24
- 3.4. Differential Evolution and Optimization Workflow . . . 25
- 3.5. Objective Function Design . . . 26
- 3.6. Experimental Setup . . . 27
- 3.7. Trial Procedure and Logging . . . 27
- 3.8. Data Analysis . . . 28
- 3.9. Research Quality . . . 28
- 3.10. Use of Artificial Intelligence . . . 29

4. Findings **31**
- 4.1. Experimental Setup . . . 31
  - 4.1.1. Robot configuration . . . 31
  - 4.1.2. Testing Environment . . . 32
- 4.2. System Identification . . . 33
  - 4.2.1. α Free Fall Test . . . 33

4.2.2. B_v Open-Loop Pulse Test . . . 34
4.2.3. B_p Why It Cannot Be Measured . . . 35
4.2.4. Consequence for LQR Design . . . 35
4.3. Hardware Fault Discovery . . . 35
4.3.1. Initial Observations . . . 35
4.3.2. Broken SPI Slave Select Pin . . . 36
4.3.3. Damaged Magnetic Encoder . . . 36
4.3.4. Impact on the Study . . . 36
4.4. Objective Function Development . . . 37
4.5. PID Optimization . . . 37
4.5.1. Phase 1 Global Search . . . 38
4.5.2. Phase 2 Local Refinement . . . 39
4.6. LQR Optimization . . . 40
4.6.1. Phase 1 Global Search . . . 40
4.6.2. Phase 2 — Local Refinement . . . 43
4.7. Comparative Summary . . . 44
4.7.1. Balancing Performance . . . 45
4.7.2. Convergence behavior . . . 45
4.7.3. Parameter space structure . . . 46

5. Analysis **47**
5.1. RQ1 . . . 47
5.1.1. Objective Function Convergence . . . 48
5.1.2. Phase 1 convergence analysis . . . 48
5.1.3. Phase 1 MAD analysis . . . 49
5.1.4. Phase 2 Convergence . . . 50
5.2. RQ2 . . . 51
5.2.1. Phase 1 convergence . . . 52
5.2.2. Phase 2 convergence . . . 53
5.3. RQ3 . . . 54
5.3.1. Balancing performance . . . 54
5.3.2. Convergence Comparison . . . 54
5.3.3. Heatmap structure comparison . . . 55

6. Discussion **57**
6.1. Theoretical Implications . . . 57
6.2. Practical implications . . . 57
6.3. Limitations . . . 58

7. Conclusions/Concluding remarks **59**
7.1. Conclusions . . . 59
7.2. Future research . . . 60

References **61**

A. LQR Heatmaps **65**
A.1. LQR Run 1 Phase 1 . . . 65
A.2. LQR Run 2 Phase 1 . . . 66
A.3. LQR Run 3 Phase 1 . . . 67
A.4. LQR Run 4 Phase 1 . . . 68

A.5. LQR Run 5 Phase 2 69

B. PID Heatmaps **71**
B.1. PID Run 1 Phase 1 71
B.2. PID Run 2 Phase 1 72
B.3. PID Run 3 Phase 1 73
B.4. PID Run 4 Phase 1 74
B.5. PID Run 5 Phase 2 75

C. System Identification **77**
C.1. Free-Fall Test Valid Results 77
C.2. Free-Fall Fit 78
C.3. B_v Open Loop Pulse Test Results 79

List of Figures

2.1. Inverted Pendulum 8
2.2. Free body diagram of a TWR 9
2.3. Structure of the PID controller 12
2.4. Magnetic sensor: AS5048 18
2.5. Motor with a magnet on its shaft 19
2.6. Ipower Motor GM 4108 20
2.7. Microcontroller: Wemos D1 R32 22

3.1. Research process overview 24

4.1. The final assembled prototype of the two-wheeled self-balancing robot 32
4.2. Test Environment 33
4.3. Mean Absolute Deviation and Low Score 39
4.4. Run 2 MAD vs Lowest f(x) 41
4.5. Run 1 MAD vs Lowest f(x) 42
4.6. Run 3 MAD vs Lowest f(x) 42
4.7. Run 4 MAD vs Lowest f(x) 43
4.8. Run 5 Convergence 44

5.1. Phase 1 best f(x) per generation 49
5.2. Phase 1 MAD Per Generation 50
5.3. Phase 2 Best f(x) per generation 50
5.4. Phase 2 MAD Per Generation 51
5.5. Phase 1 Best f(x) per generation (LQR) 52
5.6. LQR Phase 2 best f(x) 53
5.7. MAD LQR Phase 2 54

A.1. LQR Run 1 Phase 1 Heatmap 65
A.2. LQR Run 2 Phase 1 Heatmap 66
A.3. LQR Run 3 Phase 1 Heatmap 67
A.4. LQR Run 4 Phase 1 Heatmap 68
A.5. LQR Run 5 Phase 2 Heatmap 69

B.1. PID Run 1 Phase 1 Heatmap 71
B.2. PID Run 2 Phase 1 Heatmap 72
B.3. PID Run 3 Phase 1 Heatmap 73
B.4. PID Run 4 Phase 1 Heatmap 74
B.5. PID Run 5 Phase 2 Heatmap 75

C.1. Free-fall fit 78

List of Tables

4.1. Table 1: Initial Search Bounds 38
4.2. PID Phase 1 Summary 38
4.3. PID Shifted Search Bounds 39
4.4. PID Phase 2 Summary 39
4.5. Final PID Parameters 40
4.6. Initial LQR Bounds 40
4.7. LQR Phase 1 Summary 41
4.8. LQR Shifted Bounds 43
4.9. LQR Final Values 44
4.10. Final Controller Comparison 45

List of Abbreviations

Abbreviation	Meaning
ARE	Algebraic Riccati Equation
BLDC	Brushless Direct Current (motor)
DE	Differential Evolution
DMP	Digital Motion Processor
DOF	Degree of Freedom
ESP32	Espressif Systems 32-bit microcontroller
FOC	Field-Oriented Control
IMU	Inertial Measurement Unit
IP	Inverted Pendulum
I^2C	Inter-Integrated Circuit (communication protocol)
JSON	JavaScript Object Notation
LQR	Linear Quadratic Regulator
MAD	Mean Absolute Deviation
MCU	Microcontroller Unit
PID	Proportional-Integral-Derivate
PWM	Pulse Width Modulation
SPI	Serial Peripheral Interface
TWR	Two-Wheeled Robot
TWSBR	Two-Wheeled Self-Balancing Robot
Wi-Fi	Wireless Fidelity

Nomenclature

Symbol	Meaning	Unit
α	Gravitational instability coefficient	$\mathrm{rad}^2/\mathrm{s}^2$
θ	Pitch angle measured from vertical	rad
$\dot{\theta}$	Pitch angular velocity	rad/s
$\ddot{\theta}$	Pitch angular acceleration	$\mathrm{rad}/\mathrm{s}^2$
X	Horizontal position of the wheelbase	m
$\dot{x}$	Horizontal velocity of wheelbase	m/s
$\ddot{x}$	Horizontal acceleration of the wheelbase	m/s^2
x_p	Horizontal position of pendulum center of mass	m
y_p	Vertical position of pendulum center of mass	m
L	Distance from wheel axle to pendulum center of mass	m
M	Total robot mass	kg
M_m	Mass of wheelbase	kg
M_p	Mass of pendulum body	kg
I_h	Moment of inertia of wheels about hub	$\mathrm{kg} \cdot \mathrm{m}^2$
I_p	Moment of inertia of pendulum about center of mass	$\mathrm{kg} \cdot \mathrm{m}^2$
G	Gravitational acceleration	m/s^2
T	Total kinetic energy	J
V	Total potential energy	J
$\mathcal{L}$	Lagrangian	J
F_n	Horizontal ground reaction force from motor torque	N
U	Control input	V
B_v	Wheel angular acceleration per unit voltage	$\frac{\mathrm{rad}}{\mathrm{s}^2}/\mathrm{V}$
B_p	Pitch angular acceleration per unit voltage	$\frac{\mathrm{rad}}{\mathrm{s}^2}/\mathrm{V}$
K_t	Motor torque constant	Nm/A
R_{motor}	Motor winding resistance	Ω

Symbol	Meaning	Unit
v_{err}	Wheel velocity error	rad/s
e_θ	Pitch angle error	rad
$f(x)$	Objective function score	—
Stab_p	Stability loop proportional gain	—
Stab_i	Stability loop integral gain	—
Stab_d	Stability loop derivative gain	—
Vel_p	Velocity loop proportional gain	—
Vel_i	Velocity loop integral gain	—
Vel_d	Velocity loop derivative gain	—
K_1	LQR pitch angle gain	—
K_2	LQR pitch rate gain	—
K_3	LQR velocity error gain	—
K_4	LQR integrated velocity error gain	—
K	LQR gain vector $[k_1,\ k_2,\ k_3,\ k_4]$	—
Q	LQR state weighting matrix	—
R	LQR control effort weighting matrix	—
P	Solution to the Algebraic Riccati Equation	—
A	State matrix of linearized system	—
B	Input matrix of linearized system	—
D_g	Mean absolute deviation at generation g	—
F	DE mutation factor	—
CR	DE crossover rate	—

1 Introduction

This chapter introduces the research area and defines the problem. It continues by outlining the purpose and research questions, addresses the scope and delimitations, and concludes with the disposition of the study.

1.1 Background

The development of robotics has come a long way, especially for mobile robots [1]. Robots that can either follow human commands, move autonomously, or run preprogrammed sequences are called mobile robots [2]. Similar trends such as mobile robot development are also occurring in computer systems, where the transition from mainframe computing via workstations to personal computers (PCs) is expected to continue with handheld devices for many applications [1]. Previously, mobile robots were operated by heavy, large, and expensive computer systems that were not portable and required to be connected via cable or wireless devices. In the modern-day mobile robots can be built with numerous motors and sensors that are controlled by inexpensive, small, and light embedded computer systems that are onboard the robot. The interest for mobile robots has increased as they no longer are perceived as toys, but as real solutions for industrial applications. There are several different types of mobile robots, they are usually classified into two different categories: service and industrial mobile robots [2]. Service robots are for example vacuuming robots, restaurant delivering robots, etc and industrial robots are for example fruit picking robots, material handling robots, etc. The simplest type of mobile robot is the wheeled mobile robot; those robots possess one or more driven wheels [1]. There may also be a passive caster wheeled and they may also have steered wheels.

The robot may have different advantages depending on the design [1]. Robots with one steered wheel and two motors have the advantage that the driving and the steering action are completely unlinked. The control software for driving straight or curves will therefore be very simple. A disadvantage of the one-wheeled robot is that the robot will not be able to turn on the spot if the wheel is not located at the center. Differential drive is one of the most used robot designs for mobile robots, which consist of two driven wheels. Within mobile robotics the two wheeled robot (TWR) is one of the least popular, because of the structural balance and control complexity [3]. The differential drive design allows the robot to be driven straight, in a corner, or turn on the spot [1]. Another advantage of differential drive robots is that the execution of driving commands requires the generation of a corresponding wheel speed for each wheel, which must be done by software. The wheels for differential drive designs are fixed and do not need to be turned as with one-wheeled robots, which simplifies the robot's mechanical design.

The main issue with TWRs is their structural balance and complexity in control [3]. The TWR are based on the inverted pendulum (IP) system, which is a nonlinear, unstable and under actuated system [4]. The IP differs from an ordinary pendulum in that the pivot point on an IP is located below the center of mass [5], [6]. To enable self-balancing, the TWRs are equipped with different types of sensors, like inertial measurement units (IMUs), to monitor and control their balance [5]. They also use

advanced control algorithms, such as feedback control to maintain their stability. But regardless of the design all the self-balancing robots have the common goal of maintaining their balance and stability.

Several studies have been conducted on stabilizing and resolving tracking problems on mobile robots, and from that work different control algorithms have been developed, e.g. Passivity, Lyapunov Theory, Adaptive Fuzzy Logic, and classical Proportional-Integral-Derivative (PID), etc. [7], [8]. The classical PID method is one of the most well-known control methods that has been used in control of numerous robots. PID controllers and other controllers are designed under the assumption that the robot's physical parameters remain constant and do not vary over time. But in real-world conditions, external disturbances may alter how the system behaves. If the parameters of the robot change and the controller is not designed to adapt to those changes, the system may respond poorly, potentially resulting in unstable or undesirable behavior.

Several control structures have been developed to adjust their behavior in response to disturbances and system uncertainties [7]. These structures are referred to as "adaptive controllers". These controllers modify their parameters in real time to maintain the desired system performance. This capability allows the system to operate as intended while remaining sturdy against a wide range of disturbances. Various types of adaptive control approaches have been proposed for controlling and stabilizing inverted pendulums.

1.2 Research Gap

Two-wheeled self-balancing robots present a control challenge due to their instability. While PID control remains the most popular control solution for these systems, conventional tuning methods such as Ziegler-Nichols rely on linear approximations and manual adjustments, resulting in controllers that perform poorly when conditions change [9]. Linear Quadratic Regulator (LQR) provides an alternative by computing optimal gains analytically from a system model [10]. The pitch angular acceleration parameter B_p, which is critical for constructing the LQR plant model and solving the Riccati equation, cannot be measured directly on a two-wheeled platform without additional instruments. The fundamental hardware constraints that determine whether LQR is practically feasible on this class of robot, and when analytical gain derivation is prevented, are not well documented in existing literature. Bio-inspired optimization methods such as differential evolution can experimentally tune parameters directly without the need of a system model. While a promising approach, defining an objective function that genuinely captures good balancing behavior, rather than allowing the optimizer to find unintended shortcuts, is a practical challenge that has received little attention in literature. How to design such a function, and what workflow is required to reliably find well-performing parameters through physical trials, remains an open question. Furthermore, no study has directly compared cascade-PID and LQR-form state-feedback parameterizations under an identical DE-based physical-trial workflow on the same hardware.

1.3 Problem Description

Two-wheeled self-balancing robots are inherently unstable and require precise controller tuning to maintain balance. Doing this manually is time-consuming and difficult, because the relationship between the PID gains and overall balancing behavior is non-linear and non-intuitive. Bio-inspired optimization offers a way to automate this process but applying it to a physical robot is not straightforward. The optimizer needs a very clear definition of what good balancing looks like, and defining the definition in a way that cannot be cheated by the optimizer is a challenge. When a model-based alternative such as LQR is considered, further difficulties emerge. Building an accurate and robust model requires physical measurements that may not be achievable with the available hardware, and the method may face mathematical limitations that make it unsuitable for this type of system. It is therefore unclear whether automated optimization can reliably produce a well-tuned controller for this robot, what is required for it to do so, and whether a model-based approach offers any practical advantage over PID tuning.

1.4 Purpose and research questions

The purpose of this thesis is to investigate what objective function design and optimization workflow are required for differential evolution to reliably tune controllers for a two-wheeled self-balancing robot through physical trials, and to compare the balancing performance and convergence behavior of a cascade-PID parameterization against an LQR-form state-feedback parameterization when both are tuned under the same DE workflow on the same hardware. Based on the purpose of this thesis, the following research questions are formulated to evaluate the effectiveness of real-time adaptive PID tuning for a two-wheeled self-balancing robot:

RQ1: What objective function design and optimization workflow are required for differential evolution to reliably find well-performing PID parameters for a two-wheeled self-balancing robot through physical trials?

RQ2: What objective function design and optimization workflow are required for differential evolution to reliably find well-performing LQR parameters for a two-wheeled self-balancing robot through physical trials?

RQ3: How do the cascade-PID and LQR-form state-feedback parameterizations compare in terms of balancing performance and parameter convergence when tuned using the same differential evolution workflow on physical hardware?

1.5 Delimitations

This thesis has several delimitations that define its scope. The study is limited by the hardware used in the robot. The computing capacity of the control board limits the complexity and speed of the control algorithms. The accuracy and sampling rate of the sensors affect the quality of the measured data. The motors also limit performance due to their response speed and torque characteristics. These hardware components are used as selected and are not optimized further. The control algorithms are implemented within a single programming environment. The thesis does not compare different

programming languages, development tools, or software frameworks. The work focuses only on the control and stabilization of a two-wheeled self-balancing robot. Other aspects of mobile robotics, such as navigation, path planning, or obstacle avoidance, are not considered. All experiments are performed indoors on a smooth and level surface. The study does not evaluate performance on uneven terrain or in outdoor environments. The robot platform is designed for educational use. The results therefore apply only to this type of two-wheeled self-balancing robot and may not apply directly to other robot designs or industrial systems.

1.6 Outline of the thesis

Chapter	Description
Chapter 1 *Introduction*	*Introduces the research area, identifies the research gap, and defines the problem. This chapter also outlines the study's purpose, research questions, scope, and delimitations*
Chapter 2 *Theoretical Framework*	*Defines the key concepts and theories that form the foundation of the study, providing a basis for the following analysis and discussion chapters*
Chapter 3 *Methodology*	*This chapter outlines the methodology considerations that form the basis of this study, including research design, data collection, data analysis, and research quality. The chosen methodology ensures a systematic approach to addressing the research questions while maintaining academic thoroughness and reliability.*
Chapter 4 *Findings*	*Presents the empirical findings from the PID and LQR optimization runs, including hardware fault discovery, system identification, per-run convergence data, surrogate landscape analysis, and the final optimized parameters for both controllers.*
Chapter 5 *Analysis*	*Answers the three research questions by interpreting the optimization results: the objective function and workflow requirements for PID tuning (RQ1), and the same for LQR-form tuning (RQ2), and a direct comparison of balancing performance and convergence between the two parameterizations (RQ3).*

Chapter	Description
Chapter 6 *Discussion*	*Discusses the theoretical and practical implications of the findings, including the DE-tuning behavior of each control structure, the transferability of the two-phase workflow, and the limitations of the study.*
Chapter 7 *Conclusions*	*Summarizes the conclusions drawn for each research question and identifies directions for future works.*

2 Theoretical framework

This chapter establishes the fundamental theoretical framework for the study by defining key terms and concepts central to the research context. It clarifies the essential theoretical principles that guide the analysis and discussion in ensuing chapters. The selected theories and models are directly linked to the study's purpose and research questions, providing a structured basis for interpreting the findings.

2.1 Two Wheeled Robots (TWR)

There are several sorts of robots, and among mobile robots, the two-wheeled robot (TWR) is one of the most common designs due to its simple differential drive design with two driven wheels [1]. Despite its widespread use among mobile robots, the TWR remains one of the least popularized types of robots [3]. Mobile robots are robots that are capable of following human commands, move autonomously, or run preprogrammed sequences [2]. The TWR is designed with two wheels put vertical on the same axis and with the inverted pendulum (IP) system, this in turn makes the robot nonlinear, unstable and under-actuated [4], [5], [11]. A nonlinear system is one in which the relationship between variables is not proportional, meaning the output does not change at a constant rate with regard to the input [12]. This design is a typical model in control system and to enable self-balancing the TWR needs to be equipped with different types of sensors, such as inertial measurements units (IMUs). The IMUs monitor the robot's orientation and movement and control its balance [5], [11]. The two wheeled self-balancing robots (TWSBR) primary goal is to maintain its balance and prevent falling over [5]. TWSBR utilizes advanced control algorithm such as feedback control, to maintain their stability. Controllers that have been studied for stabilizing and tracking purposes include e.g. Passivity, Lyapunov Theory, Adaptive Fuzzy Logic, and classical Proportional-Integral-Derivative (PID), etc [7], [8]. In such systems, data acquired from IMU sensors is processed by the control algorithm, which can be implemented on an onboard microcontroller such as Arduino, ESP32, etc. [5]. The TWSBR maintains its balance and prevents falling over by continually monitoring the orientation with the help of the sensors. Based on the sensor data, the controller computes the necessary adjustment to the robot's movements to ensure continuous balance. Regardless of the design, all the self-balancing robots share the common goal of maintaining their stability.

2.2 Inverted Pendulum

The inverted pendulum (IP) system is a common model used to represent control problems encountered in the initial launch stages of missiles and rockets [13]. The IP, also designated as the reverse pendulum (RP), is inherently unstable and requires an external force to maintain its balance [5]. The IP system is a well-identified benchmark problem that presents several challenges to control design [4]. The system is nonlinear, unstable, non-minimum phase, and under-actuated. It is one of the most well-known problems in control theory [1], [4], [7]. The IP model has been applied in bipedal walking research, and most balancing robots are based upon the IP principle [1]. The dynamics of the system are advantageous in that they can be constrained to two dimensions, and the cost of producing an IP robot

is relatively low due to the small number of moving parts. In a traditional pendulum the pivot point is located above the center of mass, but in an IP the pivot point is located below the center of mass, as illustrated in Fig. 2.1. This design is intentional, as it makes the structure inherently unstable; the robot will fall over unless external forces are applied to counteract any tilting motion [5]. The IP is essential for balancing robots as it requires the system to continuously monitor its orientation through sensor data and apply corrective forces to remain upright. This is achieved using control algorithms such as Passivity, Lyapunov Theory, Adaptive Fuzzy Logic, and classical Proportional-Integral-Derivative (PID) control [7], [8].

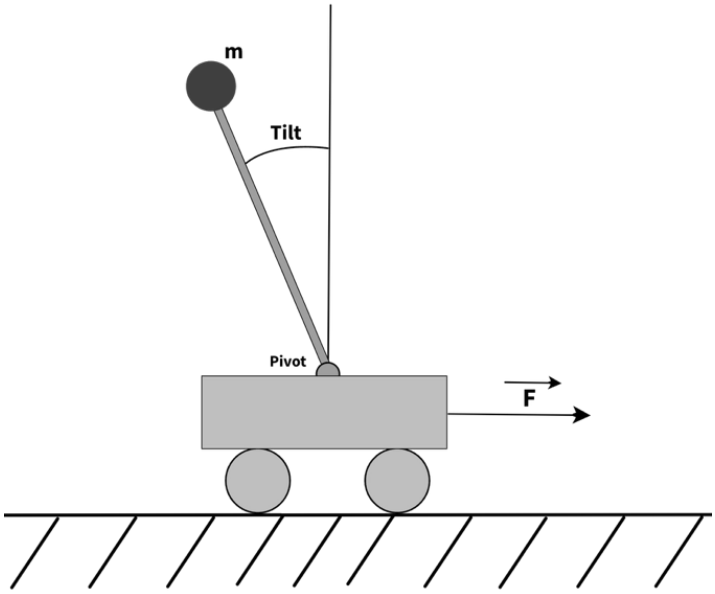


Figure 2.1.: Inverted Pendulum

Since the 1950s, the IP has been used as a benchmark system for teaching and developing linear feedback control theories. It has been used to study the stabilization of open-loop unstable systems, as well as to investigate linear optimal control and complex nonlinear control problems for unstable systems [14]. An open-loop unstable system is a system that will become increasingly unstable in the absence of corrective control actions. The IP has become a standard for evaluating and comparing different control theories over the last fifty years.

The motion of the IP can be described using various modeling approaches. Where one of the most commonly used methods is the Lagrange method, which derives the equation of motion based on the difference between the kinetic energy and the potential energy of the IP [15]. The approach provides a systematic framework for modeling the dynamics of the IP and forms the foundation for the design and analysis of many control algorithms [14].

2.2.1 Lagrange Method

In order to calculate the Lagrange method, the kinetic energy and the potential energy of the IP on a TWR need to be calculated. The following is the free body diagram of a TWR:

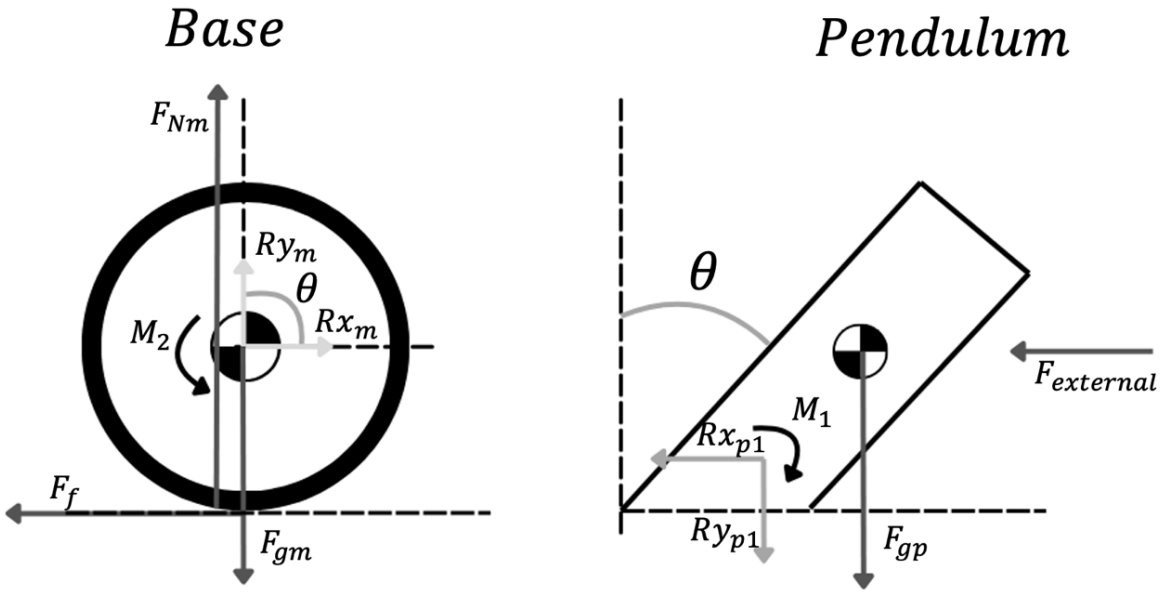


Figure 2.2.: Free body diagram of a TWR

The Lagrangian ($\mathcal{L}$) is defined by:

$$\mathcal{L} = T - V$$

The equation of motion is formulated as follows:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_i, \quad i = 1, 2$$

In order to calculate the kinetic energy of the pendulum part of the robot, the center of mass velocity needs to be calculated. The pendulums center of mass coordinates are given by:

$$x_p = x + L\sin\theta, \quad y_p = L\cos\theta \tag{2.1}$$

To obtain the velocity, the coordinates need to be differentiated with respect to time:

$$\dot{x}_p = \dot{x} + L\dot{\theta}\cos\theta, \quad \dot{y}_p = -L\dot{\theta}\sin\theta \tag{2.2}$$

The kinetic energy for the wheelbase (T_{wheel}) is then given by:

$$T_{\text{wheel}} = \frac{1}{2} M_m \dot{x}^2 + \frac{1}{2} I_h \left(\frac{\dot{x}}{R} \right)^2 = \frac{1}{2} \left(M_m + \frac{I_h}{R^2} \right) \dot{x}^2 \tag{2.3}$$

The pendulums kinetic energy is then:

$$T_{\text{pendulum}} = \frac{1}{2} M_p \left(\dot{x}_P^2 + \dot{y}_p^2 \right) + \frac{1}{2} I_p \dot{\theta}^2 \tag{2.4}$$

The following expression is achieved after inserting Equation 2.2 and simplifying the expression:

$$T_{\text{pendulum}} = \frac{1}{2} M_p \left(\dot{x}^2 + 2\dot{x} L \dot{\theta} \cos\theta + L^2 \dot{\theta}^2 \right) + \frac{1}{2} I_p \dot{\theta}^2 \tag{2.5}$$

The total kinetic energy (T) is then achieved by the sum of Equation 2.3 and Equation 2.5, which gives the following expression:

$$T = T_{\text{pendulum}} + T_{\text{wheel}} = \frac{1}{2} \left(M_m + M_p + \frac{I_h}{R} \right) \dot{x}^2 + M_p \dot{x} L \dot{\theta} \cos\theta + \frac{1}{2} \left(M_p L^2 + I_p \right) \dot{\theta}^2 \tag{2.6}$$

The first addend represents the total translational energy, the second addend corresponds to the mechanical connection between the base's movements and the pendulum's rotation, and the third addend corresponds to the pendulum's rotational energy around its own center of mass. The potential energy depends only on the pendulum's height, and the following is the expression of the potential energy (V) for the TWR:

$$V = M_p g y_p = M_p g L \cos\theta \tag{2.7}$$

Where the baselevel for the TWR is at $\theta = 90°$, which correspond to the robot's upright position. There are also forces that need to be considered, such as steering torque from the motor and damping. To take these forces into account, the principle of virtual work is considered, which is a method that defines the virtual work δW as the work done by the non-conservative forces at a small displacement of the system's generalised coordinate. Forces such as friction counteract the velocity $\dot{x}$, which performs the work $-b\dot{x}\delta x$. Damping at the pivot point also counteracts the angular velocity $\dot{\theta}$, which performs the work $-b\dot{\theta}\delta x$. The total equation is then:

$$\delta W = \frac{u}{R} \delta x - u \delta\theta - b_x \dot{x} \delta x - b_\theta \dot{\theta} \delta x \tag{2.8}$$

The term $\frac{u}{R}\delta x$ is the work done by movement at the base by the wheels, and the other terms describe the moment applied to the pendulum in the opposite direction. The equation shows how the translation of the base relates to the pendulum's rotation. Using the generalized forces Q_j with the relation $\delta W = \sum Q_j \delta q_j$, the system in this thesis has only two coordinates, x and θ, which gives the equation:

$$\delta W = Q_x \delta x + Q_\theta \delta\theta \tag{2.9}$$

In order to find Q_x and Q_θ the terms, in Equation 2.9, are collected based on if they belong to δx or $\delta\theta$:

$$\delta W = \left(\frac{u}{R} - b_x\dot{x}\right)\delta x + \left(-u - b_\theta\dot{\theta}\right)\delta\theta \tag{2.10}$$

The generalized forces are then:

$$Q_x = \frac{u}{R} - b_x\dot{x}, \quad Q_\theta = -u - b_\theta\dot{\theta} \tag{2.11}$$

Where b_x and b_θ are the damping coefficients for translation and rotation, respectively. The Lagrange operation is done for the two coordinates and for the base $q_1 = x$ gives:

$$\frac{d}{dt}\left(\frac{\partial\mathcal{L}}{\partial\dot{x}}\right) - \frac{\partial\mathcal{L}}{\partial x} = Q_x \tag{2.12}$$

The length L doesn't depend on x and that means that $\frac{\partial\mathcal{L}}{\partial x} = 0$ and the derivation of Equation 2.6 gives:

$$\frac{d}{dt}\left(\frac{\partial\mathcal{L}}{\partial\dot{x}}\right) = \left(M_m + M_p + \frac{I_h}{R^2}\right)\ddot{x} + M_pL\left(\ddot{\theta}\cos\theta - \dot{\theta}^2\sin\theta\right) \tag{2.13}$$

Equation 2.11 is inserted, and it gives the following equation:

$$\frac{u}{R} - b_x\dot{x} = \left(M_m + M_p + \frac{I_h}{R^2}\right)\ddot{x} + M_pL\left(\ddot{\theta}\cos\theta - \dot{\theta}^2\sin\theta\right) \tag{2.14}$$

For the pendulum's equation, $q_1 = \theta$ gives:

$$\frac{d}{dt}\left(\frac{\partial\mathcal{L}}{\partial\dot{\theta}}\right) - \frac{\partial\mathcal{L}}{\partial\theta} = Q_\theta \tag{2.15}$$

And the derivative gives the following equation:

$$\frac{d}{dt}\left(\frac{\partial\mathcal{L}}{\partial\dot{\theta}}\right) = \left(I_p + M_pL^2\right)\ddot{\theta} + M_pL\ddot{x}\cos\theta - M_pL\dot{x}\dot{\theta}\sin\theta \tag{2.16}$$

And the derivative of $\mathcal{L}$ with respect to θ:

$$\frac{\partial\mathcal{L}}{\partial\theta} = -M_pL\dot{x}\dot{\theta}\sin\theta - M_pgL\sin\theta \tag{2.17}$$

Equation 2.11 is inserted, and it gives the following equation:

$$-u - b_\theta\dot{\theta} = \left(I_p + M_pL^2\right)\ddot{\theta} + M_pL\ddot{x}\cos\theta - M_pgL\sin\theta \tag{2.18}$$

Equation 2.14 and Equation 2.18 give the connection between the two nonlinear motion equations. Equation 2.14 and Equation 2.18 give the reference point for several control theories on an IP. At small angular displacement the equations are approximated to:

$$\sin\theta \approx 0, \quad \cos\theta \approx 1, \quad \dot{\theta}^2 \sin\theta \approx 0$$

Which gives the approximated equations:

$$\frac{u}{R} - b_x\dot{x} = \left(M_m + M_p + \frac{I_h}{R^2}\right)\ddot{x} + M_p L\ddot{\theta}$$

$$-u - b_\theta\dot{\theta} = \left(I_p + M_p L^2\right)\ddot{\theta} + M_p L\ddot{x} + M_p gL\theta$$

2.3 Proportional, Integral, Derivate (PID)

A Proportional, Integral, Derivative (PID) controller adjusts the control signal based on three error components: proportional (P), integral (I), and derivative (D) [16]. Each component influences the system's performance in a different way. By tuning these components, the dynamic behavior of the system can be optimized to reduce overshoot, improve stability and shorten settling time.

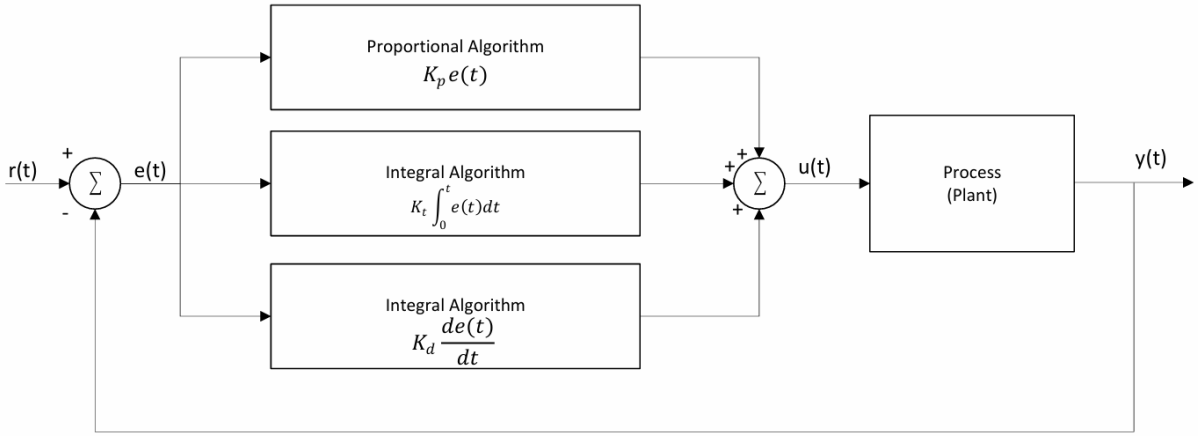


Figure 2.3.: Structure of the PID controller

The PID controller processes the current error signal $e(t)$, which is defined as the difference between the reference input $r(t)$ and the output $y(t)$ [16]. The proportional term reacts to the current error, the integral term accounts for the accumulated error over time, and the derivative term responds to the rate of change of the error. Together these components drive the system output toward the desired reference value.

Generally, the PID controller rule is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(t)\,dt + K_d \frac{de(t)}{dt} \tag{2.19}$$

$$e(t) = r(t) - y(t) \tag{2.20}$$

The double PID controller concept is used for fast rejection of disturbances before they are propagated to other parts of the system; this concept is called "cascade PID" [17]. Several researchers have shown that the cascade PID scheme has the potential of offering better performance than the classical feedback controller. The cascade PID is formed by linking the output of one controller to the setpoint of another controller, where each controller monitors different variable of the same process [18]. The primary controller generates the signal for the secondary controller, effectively coordinating the overall control action through a remote setpoint signal. Cascading PID have been applied in several fields [19], [20]. In drone system, one controller has been implemented for regulating position, while another controls the angular rate movement. This control structure enhances the stability and reliability of the system. In robots, the primary PID controller has for example been used for generating an optimal reference rotational speed and the secondary has been used as the rotational speed tracking executor [20].

2.3.1 Limitations of PID tuning

There is no universal PID tuning method that works across all systems. Instead, many tuning rules exist, and tuning remains system- and context-dependent [9]. Furthermore, PID controllers are usually tuned for a single operating point resulting in performance degradation when system parameters change. Common tuning methods, such as Ziegler-Nichols method, rely on linear approximations and simplified representations of system dynamics [9]. These assumptions often lead to large overshoot, oscillations, and the need for manual retuning when operating conditions vary. As a consequence, fixed-gain PID controllers struggle with non-linear and unstable systems, particularly systems with fast dynamics or structural instability.

Furthermore, traditional PID controllers also use constant parameters that cannot adapt to changing system behavior or external disturbances [16]. This heavily limits robustness and reduces control accuracy in dynamic environments. Additionally, the derivative term can amplify measurement noise, which can negatively affect stability if sensor signals are noisy. As control tasks demand higher precision and faster response, these limitations reduce the suitability of conventional PID control for complex and time-varying systems.

2.4 Linear Quadratic Regulator (LQR)

The linear quadratic regulator (LQR) was first proposed by Rudolf Kalman in 1960 and later used by Richard Bellman on an IP in order to develop the LQR control theory, which is a control theory used in several fields such as structural control engineering, vibration suspension, robotics, etc. [10], [21], [22], [23]. LQR is a common controller used in the stabilization of the IP problem, as it provides a robust control with feedback [7], [10], [24]. The controller consists of computing a quadratic cost function where quadratic terms for state and control variables are incorporated in the function. LQR has the possibility of weighting matrices in which the system performance is shaped by these matrices. This enables systematic prioritization of state variables and control input to achieve the desired performance. The traditional LQR controller requires solving the Riccati equation, a nonlinear ordinary differential

equation that has an essential role in control theory. The LQR controller aims to minimize the cost function J which is defined as following:

$$J = \int_0^\infty (x^T Q x + u^T R u + 2x^T N u)\, dt \tag{2.21}$$

Where x^T is the performance and u^T is the effort of actuators, Q is an $[n \times n]$ matrix and R is an $[m \times m]$ matrix, and they are the weighting matrices that can vary depending on the desired performance [25], [26]. The R is the matrix coefficient that determines the weight of energy cost, and the Q matrix is the weight coefficient determining the importance of accuracy. Then x and u are the state vector for each quality, if the state vector is zero then the state is aiming to get back to its starting equilibrium point. The T is the process completion time. The matrices Q and R correspond to each actuator's performance and effort, and these matrices have the same number of rows as states. They are also positive definite so that the product of the matrices and states vectors are positive and non-zero. The N matrix helps to penalize cross product of x and u, and if the matrix is set to zero then the matrix is omitted. Then the cost function is then shorter and looks like the following equation:

$$J = \int_0^\infty (x^T Q x + u^T R u)\, dt \tag{2.22}$$

Which can be rewritten as the following equation:

$$J = \int_0^\infty \left(\begin{bmatrix} x^T & u^T \end{bmatrix} \begin{bmatrix} Q & 0 \\ 0 & R \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix} \right) dt \tag{2.23}$$

There is value zero in the matrix when the value of N is zero. The cost function J as shown in the equation above, is then integrated and the integrations gives the following equation:

$$u = -Kx \tag{2.24}$$

Where K is the optimal gain matrix in the state feedback controller and it is a $[m \times n]$ matrix that consist of free constant coefficients. The constant K is given by $K = R^{-1}B^T P$, where the P matrix must fulfill the Riccati equation:

$$A^T P + PA + PBR^{-1}B^T P + Q = 0 \tag{2.25}$$

The LQR is sensitive to changes in parameters, making it necessary to apply optimization methods to tune the gains so that system stability and optimal performance are maintained despite such changes [10]. The controller also requires precise knowledge of the system model, such as knowledge of the mass placed on the pendulum [21], [27]. It is also a challenge in the selection of weighting the matrices Q and R [10]. The standard approach relies on trial and error, for example, assigning large Q matrices to penalize deviation from reference states. This approach becomes tedious in higher order systems which result in using optimization techniques such as Particle Swarm Optimization (PSO), Genetic Algorithms (GA), Artificial Bee Colony (ABC), and Differential Evolution (DE).

In this thesis, B_{pitch} cannot be measured directly on the available hardware, which means the Riccati equation cannot be solved. The LQR control structure $u = -Kx$ is therefore retained, but the gain vector K is found using differential evolution optimization over physical trials rather than derived analytically. This is described in Section 3.4.

2.5 Optimization

Optimization is the process of systematically searching for the values of a set of variables that minimize or maximize some objective function. In engineering the objective function usually represents a performance criterion, such as energy consumption, error magnitude, or response time, and the variables are parameters of a system that can be adjusted to improve the criterion [28].

Formally, an optimization problem can be written as:

$$\min_{x \in X} f(x)$$

Where $x \in \mathbb{R}^n$ is the vector of variables, $f : \mathbb{R}^n \to \mathbb{R}$ is the objective function, and X defines the feasible region, typically expressed as a set of bounds $x_l \leq x \leq x_u$.

2.5.1 Classical Optimization Methods

Classical methods rely on the mathematical properties of the objective functions to guide the search. Gradient-based methods compute or approximate the gradient $\nabla f(x)$ and follow it toward a minimum. This is highly efficient but requires the objective function to be differentiable and is susceptible to converging to a local minimum rather than the global optimum [28]. For convex problems, where any minimum is the global minimum, gradient methods are generally preferred. For non-convex problems, very commonly found in engineering, the choice of starting point heavily influences the solution found.

2.5.2 Metaheuristic Optimization

When the objective is non-differentiable or discontinuous, classical gradient-based methods are not applicable. Metaheuristic methods solve this problem by treating the objective function as a black box, requiring only the ability to evaluate $f(x)$ at any given point without needing analytical expressions or derivatives [29]. Metaheuristic methods are divided into single-solution methods, which maintain and iteratively improve one candidate solution, and population-based methods, which maintain a set of candidate solutions and use interactions between them to guide the search. The population-based methods often offer stronger global search capabilities because the population provides simultaneous coverage of multiple regions of the search space, reducing the risk of premature convergence to a local minimum [30]. A prominent class of population-based metaheuristic optimization is evolutionary algorithms. They maintain a population of candidate solutions across successive generations, applying variation operators such as mutation and crossover to generate new candidates, and selection to keep the better performing ones. Over generations, the population concentrates around regions of the search space associated with good objective values [30]. The population size and mutation rate are key design parameters that govern the balance between exploration of new regions and exploitation of known good regions, a fundamental trade-off in all optimization algorithms [29].

2.6 Differential Evolution

Differential Evolution (DE) is an optimization algorithm first proposed by Storn and Price (1997) [31]. Unlike gradient based methods, DE does not require the objective function to be differentiable or even continuous, making it well suited to noisy, non-analytical problems where the relationship between input parameters and output performance cannot be expressed as a closed-form mathematical function. DE uses a population of candidate solutions, each representing a vector of parameter values. At each generation the algorithm applies three operations to evolve the population toward better solutions.

Mutation — for each candidate vector x_i in the population, three randomly selected vectors x_a, x_b, x_c are chosen and a mutated vector is formed:

$$v_i = x_a + F \cdot (x_b - x_c) \tag{2.26}$$

where F is a mutation factor that controls the magnitude of deviation. The difference vector $(x_b - x_c)$ provides a step size that naturally scales with the spread of the population, large early in the search when candidates are spread widely across the parameter space, and smaller later when the population has converged around a promising region [31].

Crossover — the mutant vector v_i is combined with the original candidate x_i elementwise with probability CR (the crossover rate) to produce a trial vector u_i:

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{if } \text{rand}(0,1) \leq CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}, & \text{otherwise} \end{cases} \tag{2.27}$$

The condition $j = j_{\text{rand}}$ ensures at least one parameter is always taken from the mutant vector, preventing the trial vector from being identical to the original candidate.

Selection — the trial vector u_i replaces the original candidate x_i in the next generation only if it achieves a lower objective score:

$$x_i^{(t+1)} = \begin{cases} u_i, & \text{if } f(u_i) \leq f(x_i) \\ x_i, & \text{otherwise} \end{cases} \tag{2.28}$$

This method of selection ensures that the population never degrades, each generation is at least as good as the previous one [32].

2.6.1 Properties Relevant to Engineering Optimization

DE possesses several properties that make it applicable to engineering optimization problems where analytical methods are unavailable or impractical. Since it only evaluates functions and not gradient information, DE is applicable to objective functions that are non-differentiable, discontinuous, or defined only through physical measurement or simulation [31]. Also, because decisions are made across an entire population rather than based on precise gradient estimates, DE tolerates a degree of measurement noise without being disturbed by noisy evaluations [33].

In addition, DE can search globally. Using the difference vector mutation, DE can explore the full parameter space rather than converging to a local minimum. This is particularly suited for multi-parameter engineering problems where the performance landscape may contain multiple local optima [32]. Finally, DE can effectively handle infeasible candidates that violate physical or operational constraints by assigning them a maximum penalty score [33].

2.6.2 Applications in Control System Tuning

DE has been used extensively to tune control system parameters. Awad et al. (2018) [34] demonstrate that DE-based optimization with informed search space definition significantly outperforms random search and gradient-based methods on engineering parameter optimization problems, especially when prior knowledge is used to narrow the search bounds across several optimization runs. This property makes DE well suited to iterative experimental tuning.

2.7 Hardware and Implementation

The TWSBR utilizes different types of sensors to gather information on the robot's orientation and movements, and the sensors are also used to monitor and control the robot's balance [5]. The TWSBR is also equipped with two wheels put vertical on the same axis and both wheels are driving wheels as the robot has a differential driven design [1], [5]. The data from the sensors are then processed by the controllers to make the necessary adjustments to the robot's movements to ensure continuous balance [5]. Regardless of the design, all the self-balancing robots share the common goal of maintaining their stability.

2.8 Sensors

The TWSBR is equipped with different sensors which fulfill different functions [5]. Sensors have traditionally been simple devices designed to convert physical quantities into electrical signals or to produce changes in electrical properties [35]. Sensors have other functionalities, such as low power consumption and being physically small. Sensors have been developed into more advanced and sophisticated systems, now serving a technically and economically practical role in most internet of things applications and even used in wheeled robots [5], [8], [35]. The sensors used in wheeled robots have different functions depending on the type of sensor, e.g. IMUs are used for monitoring and controlling the balance of the robot, and rotation angle sensors are used to measure the angles of the motors [5], [36].

IMUs are position sensors that are used in TWR to monitor and control the balance of the robot [5], [37]. An IMU is an internal sensor system for measuring changes in a robot's orientation, and this sensor system automatically provides position and orientation data [1], [37]. An IMU usually consists of three accelerometers and three gyroscopes, one for each axis, along with an onboard processor that processes the sample data [1]. Accelerometers are compact devices that measure motion intensity over time, and gyroscopes are sensors that detect rotational motion and orientation to identify the direction an object is facing [38], [39].

Adafruit ICM-20948 is one type of IMU that consist of three axis gyroscopes, three axis accelerometers, and three axis compass [5], [37]. Adafruit ICM-20948 all nine measurement axes are captured through a set of 16-bit analog-to-digital converters, which precisely translate the raw analog signals from the MEMS sensors into digital data, accessible via the I²C interface. Where I²C is a serial communication protocol, so data is transmitted over the SDA (Serial Data) line [40].

Magnetic sensors are used in the field of factory automation, where the magnetic sensors are attached to industrial robots for the purpose of precision positioning [36]. The sensors are excellent to detect the angle accuracy, and magnetic sensors are resistant to dirt and dust compared to other sensors such as optical sensors. Magnetic sensors consume less power, and they are also smaller in size and have a lower cost compared to optical sensors. AS5048 (see Fig. 2.4) is a magnetic encoder, which is a type of magnetic sensor that only detects rotational position information as changes of the magnetic field [41], [42]. The encoder then converts them into electrical signals and outputs the electrical signal. The simplest encoder consists of a magnet and a magnetic sensor. The magnet is usually attached to a rotating body such as motor shaft (see Fig. 2.5), and the magnetic sensor is fixed on a PCB board at a position such that it receives the magnetic field generated by the magnet that is attached to the rotating body. When the magnet that is attached to the motor shaft rotates, the direction of the magnetic field that is detected by the magnetic sensor changes, as a result the encoder can then detect the rotational position and speed of the motor shaft.

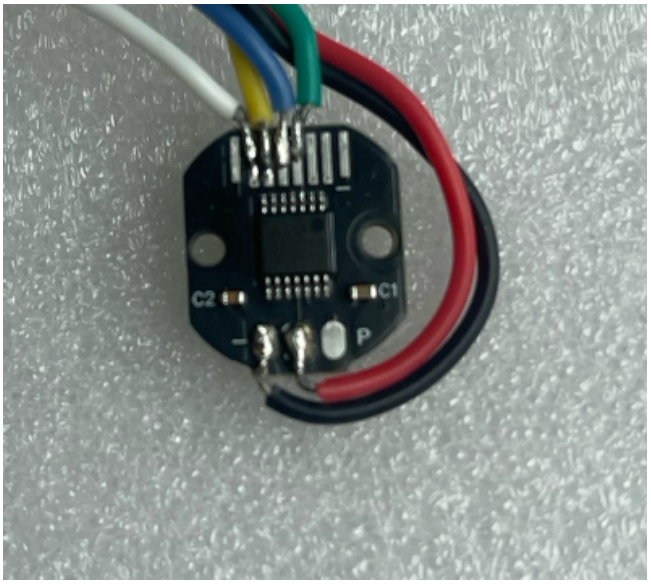


Figure 2.4.: Magnetic sensor: AS5048



Figure 2.5.: Motor with a magnet on its shaft

2.9 Motors

TWSBR is a type of robot that utilizes two driven wheels that are placed vertically on the same axis [1], [5]. The wheels are connected to motors which makes each wheel driven, and the precise balance of the robot is achieved with precise motors. There are different ways that robotic actuators can be built, where actuators are components that enable movement by converting energy, such as electrical energy, into mechanical force [1], [43]. Most notably, these actuators include electric motors, and these motors can be direct current (DC) or alternative current (AC), brushed or brushless [1].

Brushed motors have brushes to commutate the motor to spin, and brushless (BL) motors replace this mechanical function with electronic control [44]. Brushed motors require periodic maintenance, as the brushes contribute to mechanical wear and increased noise. In contrast, BL motors exhibit reduced mechanical wear and lower noise levels. However, BL motors are typically more expensive due to the need for more complex control electronics compared to brushed motors. The iPower GM4108H-120T (Fig. 2.6) is a three-phase BLDC outrunner gimbal motor used in this thesis, with two units driving one wheel each. The motor was selected for its smooth torque characteristics and suitability for direct-drive low-speed applications. It is driven by a SimpleFOC shield connected to the ESP32 microcontroller.



Figure 2.6.: Ipower Motor GM 4108

2.9.1 Motor Driver

The BLDC motor is a type of motor that employs electronic commutation instead of mechanical commutation [44]. Its stators usually consist of three-phase windings; the stator is the stationary component in the motor through which energy flows from or to the rotor, and the rotor contains permanent magnets [45], [46]. Unlike the brushed DC motors, the absence of mechanical commutation prevents the motor from independently reversing the current in the windings [45]. Consequently, applying a DC supply directly to the stator windings generates only a stationary magnetic field. Under these conditions, the rotor may align with the magnetic field or exhibit minor oscillations, but continuous rotation cannot be achieved. Therefore, a BLDC motor needs to rely on a motor driver to perform the commutation function that was previously performed by mechanical components. The motor drivers' main functions are to determine which stator phase should be energized based on the rotor position, regulate the voltage and current applied to the windings to control motor speed and output torque. The motor driver functions as a three-phase inverter, converting DC input into three phase current with controllable amplitude and phase. The generated three phase currents produce a rotating magnetic field in the stator, which continuously drives the rotor and enables sustained motor rotation.

2.10 Microcontroller

Microcontrollers are small integrated circuits built to control a specific task within an embedded system, combining a processor, memory, and input/output (I/O) components all in one chip [47]. In a TWSBR the microcontroller is the central processing unit responsible for reading the sensor data, running the control algorithms, commanding the motor drivers and handling wireless and analog communication, all simultaneously [1], [5], [7], [8]. The microcontroller also determines the necessary adjustment for the robot so it can maintain balance. Several types of microcontrollers are used in different applications such as automobile engine systems, home appliances, robots, among other devices. Depending on the application different type controllers may be used, Raspberry Pi microcontrollers may be used in complex and high-level task such as image processing and sensor data evaluation, it is not ideal for driving a robot as it misses crucial hardware components [1]. Arduino controllers are a good choice when choosing low-cost hardware, as the Arduino Nano is ideal for small projects, and for larger, more complex projects the Arduino Uno or Arduino Mega are a better choice as they can accommodate add-on boards or shields. Disadvantages of Arduino boards are their relatively low processing power, limited memory capacity that reduces their speed and efficiency [48]. They are also less suitable in real-time applications because they depend on operating system that does not support real-time monitoring. There are a number of small single-chip embedded microcontrollers more powerful than Arduino; the ESP32 family of controllers is one of the most prominent sets of processors [1]. The ESP32 microcontrollers are compatible with the Arduino library, and this allows Arduino code to run on the ESP32, enabling the controller to implement the simple field-oriented control (SimpleFOC) software and hardware [1], [49].

The specific board used in the study was the Wemos D1 R32 (see Fig. 2.7), which is a microcontroller board based on the ESP32; the board is a powerful microcontroller that has built-in Wi-Fi and Bluetooth [50], [51]. The ESP32 is a low-cost and energy efficient board, the board includes components that are essential for wireless communication, such as integrated antenna switches. As the microcontroller is compatible with Arduino, it enables the SimpleFOC Shield motor driver to be stacked directly on top of the board without additional wiring, reducing the potential points of failure [49]. The microcontroller is an essential electrical component for self-balancing robots as the component is the one that instructs the motor driven wheels on how to rotate the wheels which helps the robot balance [52], [53].



Figure 2.7.: Microcontroller: Wemos D1 R32

2.10.1 SimpleFOC

Simple field-oriented control (FOC) is a library for controlling BLDC and stepper motors; it is a well-known strategy for controlling different types of electrically commutated motors [54]. BL motors rely on control algorithms and electronics to create the necessary magnetic fields and ensure the motors' desired motion. Several techniques have been developed where FOC is one of the most efficient, but the FOC has a complex control architecture resulting in requiring substantial computational performance. The FOC implementation is almost exclusive to certain microcontroller architectures, motor drivers, current and position sensors, and motors. The ESP32 is one of the supported microcontrollers; magnetic sensors and BLDC motor drivers are also supported, and the motion control from SimpleFOC has been implemented in BLDC motors. Motion control can be implemented in numerous strategies of motion, torque, and position control. The SimpleFOC library provides a more generic and easier to use implementation of the FOC method, enabling the rapid development of highly dynamic cyber physical system such as control theory experimental setup.

3 Methodology

This chapter outlines the methodology considerations that form the basis of this study, including research design, data collection, data analysis, and research quality. The chosen methodology ensures a systematic approach to addressing the research questions while maintaining academic thoroughness and reliability.

3.1 Research Design

Using a well-constructed research design increases the validity, reliability and generalization of the findings, while also providing a structured roadmap on data collection, analysis and interpretation [55], [56], [57]. This thesis employs a quantitative, experimental, and comparative methodology to explore how well the cascaded PID and LQR-form state-feedback controllers perform on a TWSBR.

The methodology consists of experimental procedures, where the different controllers are tuned through identical DE-based physical trial workflow on the same robot. By maintaining a controlled environment, the precision can be maximized and the drawing of specific conclusions can be drawn, ensuring that the observed differences in performance can be attributed to the control methods themselves [58]. The study also conducted comparative procedures with the data gathered from the experiments, which enable a direct comparison of the convergence and final balancing performance from each controller. The study was conducted over twenty weeks with full-time involvement from two master students.

3.2 Research Process

The research process consisted of several stages and can be summarized as follows: problem identification related to the TWSBR, formulating the research aim and RQs, conducting a literature review, physical robot development, experimental design, controller implementation, experimental testing, data collection, data visualization, and data analysis.

More specifically, the research process began with identifying the balancing problem associated with the TWSBR, followed by the formulating of the purpose and RQs. A profound literature review was then conducted to establish a deeper understanding of TWSBR systems and relevant control methods. The next step was to build a physical TWR, and following the building phase, the selected controllers were implemented on the physical robot and evaluated throughout a series of experiments. Data collection, data visualization, and data analysis were carried out iteratively. Finally, the findings, analysis, discussion, and conclusion were completed. See Fig. 3.1 for an illustrative overview of the research process.

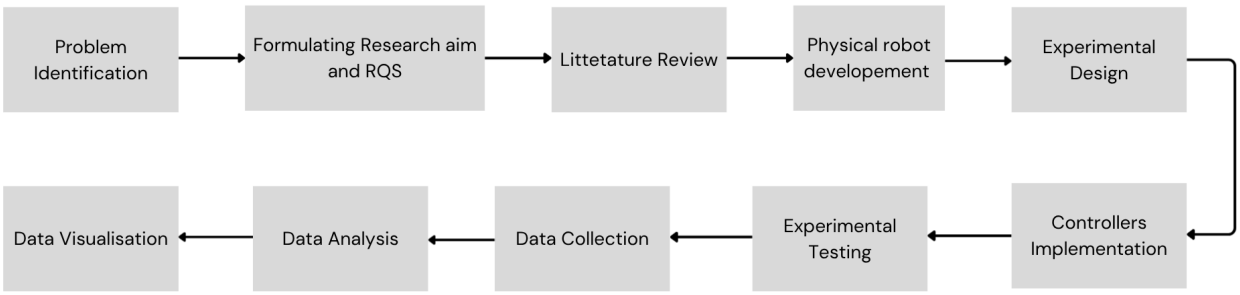


Figure 3.1.: Research process overview

3.3 Control Architectures

The robot uses a cascade PID control structure with two coupled loops. The inner stability loop regulates pitch angle using a PID controller with gain stab_p, stab_i, and stab_d. The outer velocity loop is responsible for regulating wheel velocity using gains vel_p and vel_i. The velocity loop output sets the target pitch for the stability loop, coupling the two controllers. Five parameters are optimized in total; vel_d was excluded as it consistently caused instability. The control law for the stability loop used in this thesis builds on the general formula presented in Section 2.3 as Equation 2.19. The control law used is:

$$u_{\text{stab}} = \text{stab}_p \cdot e_\theta + \text{stab}_i \cdot \int e_\theta \, dt + \text{stab}_d \cdot \dot{e}_\theta \tag{3.1}$$

Where e_θ = target pitch − measured pitch.

The LQR-form controller uses a single state-feedback law:

$$u = -\left(k_1\theta + k_2\dot{\theta} + k_3 v_{\text{err}} + k_4 \int v_{\text{err}}\right) \tag{3.2}$$

With the state vector:

$$\mathbf{x} = \begin{bmatrix} \theta & \dot{\theta} & v_{\text{err}} & \int v_{\text{err}} \end{bmatrix}^T \tag{3.3}$$

Four parameters are optimized in total. As described in Section 2.4, B_{pitch} cannot be measured on this platform, so the gain vector K is found through DE optimization over physical trials rather than derived analytically from the Riccati equation.

3.4 Differential Evolution and Optimization Workflow

Parameter optimization for both controllers was performed using Differential Evolution as implemented in the SciPy scientific computing library [59]. DE is a population-based evolutionary algorithm well suited to noisy, non-differentiable objective functions such as physical robot measurements [31]. DE uses a population of candidate parameter sets and mutates them, retaining candidates that improve the objective score and discarding those that do not [32]. The following configuration was used throughout all optimization runs:

Parameter	Value
Population size	8
Generations	50
Candidates per generation	40, 32 (8×5 parameters for PID, 8×4 for LQR)
Trials per run	~2000

The optimization was conducted in two phases. Phase 1 consisted of four independent DE runs with identical bounds but different random seeds (42, 67, 1337, 11). Using multiple seeds guards against premature convergence to a local minimum and allows the consistency of results across different starting conditions to be assessed. After all phase 1 runs were completed, surrogate landscape heatmaps were generated for each run by fitting a radial basis function interpolator to the sampled trials and visualizing pairwise parameter projections. These heatmaps were used to assess whether the optimizer had converged within the search space or was pushing against a bound boundary. When the best solutions clustered against a bound wall, the bounds were shifted in that direction for phase 2. Phase 2 consisted of a single DE run with bounds shifted and tightened around the region identified from the phase 1 heatmaps, using the same configuration as phase 1.

3.5 Objective Function Design

The objective function describes what the optimizer is minimizing, in the case of this research it is the mathematical representation of "good balancing". Designing the function is non-trivial because a poorly defined objective function allows the robot to find unintended solutions that score well without actually balancing. Four iterations of the objective function were developed throughout testing.

Version 1 — Peak-to-Peak Amplitude

$$\text{score} = \text{pitch}_{\max} - \text{pitch}_{\min}$$

This function is very simple and sought to minimize the pitch angle over the measurement window. This formulation was immediately exploited as the robot tended to lean against the walls of the test enclosure, producing near-zero oscillation while not balancing at all.

Version 2 — Amplitude with Lean Penalty

$$\text{score} = \left(\text{pitch}_{\max} - \text{pitch}_{\min}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i|$$

A mean absolute pitch term was added to penalize sustained lean away from upright. The weight of 2 makes this term twice as influential as oscillation, reflecting that a robot leaning at a consistent angle is fundamentally not balancing regardless of how little it oscillates. This partially addressed the wall-leaning issue but did not fully resolve it when the lean angle was small.

Version 3 — Amplitude, Lean, and Inverse Velocity Penalty

$$\text{score} = \left(\text{pitch}_{\max} - \text{pitch}_{\min}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i| + \begin{cases} \frac{0.5}{|\bar{v}|+0.1} & \text{if } |\bar{\theta}| > 5^\circ \\ 0 & \text{otherwise} \end{cases}$$

A key insight here was that a wall-leaning robot is stationary, it has near-zero velocity. Therefore, an inverse velocity term was added to penalize stillness, conditional on the robot also leaning. The condition prevented penalizing a genuinely stable upright robot that makes small corrective movements. However, this version introduced a new unintended behavior: the robot discovered that drifting freely across the enclosure produced high velocity, which was now being rewarded rather than penalized.

Version 4 — Final Formulation

$$\text{score} = \left(\text{pitch}_{\max} - \text{pitch}_{\min}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i| + e^{|\bar{v}|} - 1 \tag{3.4}$$

$$\text{if } \max\left(|v_i|\right) > 1.5\,\text{rad/s} \Rightarrow \text{score} = 999$$

The velocity logic was inverted — high velocity is now penalized exponentially. The exponential function $e^x - 1$ was chosen because it is exactly zero at rest, grows slowly for small corrective movements

characteristic of good balancing, and grows aggressively for larger velocities characteristic of drifting or instability. A harsh threshold of 1.5 rad/s was introduced as a runaway detection mechanism, returning the maximum penalty score and completely excluding these candidates from future generations.

3.6 Experimental Setup

The test platform is a TWSBR robot driven by two BLDC motors with a wheel diameter of 40 mm, controlled by an ESP32 microcontroller running the SimpleFOC library for motor control. Power is supplied by a 3-cell LiPo battery with a nominal voltage of 11.1V and maximum charge of 12.6V. Total mass with battery is 0.753 kg. Pitch angle is measured by an Adafruit ICM-20948 IMU mounted on top of the robot [60]. On every boot the IMU performs an automatic calibration routine requiring the robot to be held still and upright. The calibration routine was kept consistent across all sessions. Following the calibration, the DMP filter is allowed to converge before motor control is enabled. Wheel velocity is measured via magnetic encoders using AS5048 sensors communicating via SPI. Two faults were identified and corrected before any valid data was collected: a broken SPI slave select pin that caused corrupted velocity readings across both encoders, and a damaged AS5048 that produced velocity spikes. All optimization runs were conducted with the robot placed inside a cardboard enclosure. The enclosure contained the robot when it fell and prevented it from rolling away during trials, while being large enough to allow free balancing without wall contact.

3.7 Trial Procedure and Logging

Trial results were logged in real time to an Excel file using the openpyxl library, with the file saved after every trial to prevent data loss. Each optimization trial followed a fixed automated procedure:

1. A WebSocket connection is established to the robot over Wi-Fi
2. Motors are enabled and the candidate parameters are transmitted
3. A settle period of 2.0 seconds is observed during which pitch samples are collected and averaged to compute a per-trial drift offset:

$$\text{pitch}_{\text{offset}} = \frac{1}{N_{\text{settle}}} \sum_{i=1}^{N_{\text{settle}}} \theta_i^{\text{settle}}$$

4. A measurement window of 3.0 seconds during which each pitch sample is corrected for drift:

$$\theta_i^{\text{corrected}} = \theta_i^{\text{raw}} - \text{pitch}_{\text{offset}}$$

5. The objective score is computed from the collected samples.
6. The WebSocket connection is closed, and results are logged to Excel.

Every trial was recorded with the following fields: trial number, timestamp, all optimized parameters, composite score, pitch amplitude, mean absolute pitch, mean velocity, and fall status.

3.8 Data Analysis

Optimization progress was assessed using two metrics per generation. The primary metric is the best $f(x)$ per generation, which is the lowest score found across all trials up to and including generation g. This directly tracks improvement of the best solution over time and is the primary basis for comparing convergence across seeds and between PID and LQR.

The secondary metric is the mean absolute deviation from the best $f(x)$:

$$D_g = \frac{1}{N} \sum_{i=1}^{N} |f_i - f_{\text{best}}| \tag{3.5}$$

Where the sum spans across all trials in generation g and f_{best} is the best score found so far. A declining D_g indicates the population is clustering around the best solution. Surrogate heatmaps were generated by fitting a radial basis function interpolator to all valid trial data, normalized to [0, 1] per parameter. Plots were centered on the best solution in raw parameter space with equal window widths on both sides, so the best point always appears at the center. Yellow regions indicate low scores; purple indicate high scores.

3.9 Research Quality

Quantitative data are defined as data that are measured and expressed in numerical values [61]. This type of data helps identify patterns, trends, relationships, and differences between variables through objective measurement and statistical analysis. Good research is characterized by evidence that is reliable, applicable across different practical settings, consistent, and objective [62].

Reliability and validity are fundamental concepts used in research methodology to assess the quality of the conducted research [63]. Where reliability refers to the consistency of data measurement or research result over time, while validity concerns whether the research measures what it claims to measure. These concepts collectively determine how effectively a methodology, data collection technique, or data analysis plan measure the intended study variables and parameters.

Assessing the accuracy and consistency of research instruments is important, as it enhances both the reliability and validity of a study by improving the precision and accuracy of the research evaluation process [63]. It is therefore important that the methodology and data collection methods are valid to produce useful results, and that the research measures what it claims to measure. This also ensures that the discussion of the result and the conclusion drawn are valid. According to Ahmed and Ishtiaq (2021) [63] validity and reliability can be enhanced through the use of various methodologies and data collection methods, which ensures that the correct and relevant information is obtained.

The objective of research reliability and validity is to ensure that the data is comprehensive, replicable, and that the results produced are accurate. Validity refers to the integrity and application of the methods used, and the extent to which the findings accurately reflect the collected data [64]. High reliability in one indicator that shows that the result is valid, whereas unreliable data-collection methods are unlikely to produce valid findings. However, although reliability and validity, are closely related

but they are not the same. To ensure validity, the element of reliability is not enough on its own, as reliable test may not precisely reflect the real situation [63].

Reliability can be measured by examining the consistency of results over time, across different observers, and among various parts of the test itself [63]. This can be achieved by comparing different versions of the same measurement. Research validity is, on the other hand, determined by how closely the study's findings align with established theories and other measures of the same concept.

Generalizability refers to the process of making broader conclusions based on specific examples, by using the observed data to make reasonable assumptions about what has not been directly studied [65]. It is a main factor in quantitative research for evaluating the quality of a study, and it is linked to external validity, which has been regarded as important standard in research. External validity refers to which findings from a study can be generalized to other people, settings, or situations.

Objectivity is an important principle indicating that scientific claims, methods, results, and researchers should not be influenced by values, particular perspectives, personal interest, etc. [66]. The research should be based on facts and evidence rather than subjective perspective. Objective can attribute to scientific measurement, the development and improvement of scientific theories, and explanations that accurately represent reality [67]. It helps ensure that research findings aren't influenced by bias, that positive results are not false positive beyond acceptable statistical limits, and that the result can be independently reproduced by other researchers.

3.10 Use of Artificial Intelligence

Artificial intelligence tools were used to assist with certain aspects of this thesis. Claude (Anthropic) was used throughout the software development, particularly for support with Python scripting, debugging, and code structuring. AI was also used to assist with the language refinement, restructuring of text, and improving the clarity, and structuring of sections in the report. All AI-generated suggestions and written material were critically reviewed, validated, and edited by the authors before inclusion in the thesis. The authors retained full responsibility for the accuracy, quality, and integrity of the final content. AI tools were used solely as supportive instruments and did not replace independent analysis or academic judgment. The main research contribution of the thesis, including the experimental work, system identification, hardware integration and validation, data analysis, interpretation of the result, and final analytical conclusions were conducted entirely by the authors.

4 Findings

This chapter presents the empirical findings from the PID and LQR optimization runs, including hardware fault discovery, system identification, per-run convergence data, surrogate landscape analysis, and the final optimized parameters for both controllers.

4.1 Experimental Setup

This section describes the experimental setup used in this thesis, including the robot configuration and the testing environment in which all optimization runs were conducted.

4.1.1 Robot configuration

The robot that was built and used during the study was inspired by previous course instructions, and the design of the robot was based on a previous robot with some changes. The wheel diameter was 40 mm, and the two wheels were placed 12.5 cm apart; the height was 235 cm, and the width 40 mm. The center of gravity was positioned higher than the previous robot that the design was inspired from and was placed above the wheel axis to create the IP design that is required for self-balancing operations.

The robot is equipped with two magnetic encoders, specifically the AS5048 magnetic encoders as described in Section 2.8, used to detect the rotational position and speed of the motor shaft. An IMU, specifically the Adafruit ICM-20948 IMU is implemented to monitor the robot's tilt orientation. The robot was also equipped with two GM4108H-120T BLDC motors on each wheel, and each motor was connected to a respective Arduino SimpleFOCShield motor driver described in Section 2.9. An ESP32 microcontroller and a LiPo battery with a nominal charge of 11.1V was used to power the robot. The controller code was written in C++, and the optimization code in Python.

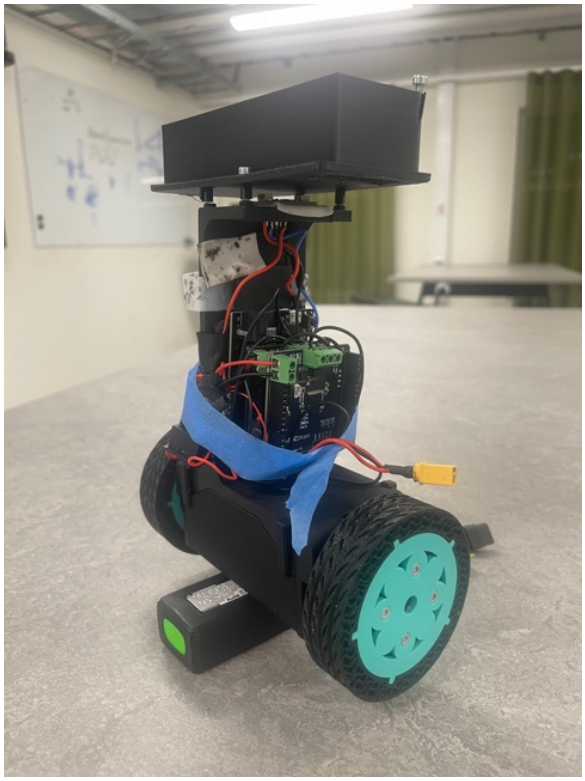


Figure 4.1.: The final assembled prototype of the two-wheeled self-balancing robot

4.1.2 Testing Environment

The test was conducted in a controlled indoor environment within a confined cardboard box (see Fig. 4.2) with the dimensions 60 cm × 40 cm and a height of 22 cm. The enclosed setup minimized external disturbances and ensured consistent testing conditions for both controllers. The test box was fitted with insulation material to protect the cardboard box from wear and tear when the optimization was performed and to minimize the risk of the robot sticking to the wall during the optimization process. The bottom of the cardboard box was also folded so that the robot could perform the optimization on a flat and smooth surface, the box was also secured to the table to minimize the movement during the optimization process of the robot.



Figure 4.2.: Test Environment

4.2 System Identification

Three physical parameters are required to construct the plant model used in the LQR investigation: α, the gravitational instability coefficient; B_v, the wheel angular acceleration per unit voltage; and B_p, the pitch angular acceleration per unit voltage. Each parameter was identified through dedicated experiments. However, as described below, B_p cannot be measured directly on this platform, which means the B matrix of the state-space model remains incomplete and the Riccati equation cannot be solved.

4.2.1 α Free Fall Test

The gravitational instability coefficient α characterizes how rapidly the robot falls when unsupported. It appears in the linearized state-space model as the unstable pole of the pitch dynamics, and its value determines the minimum gain required for a proportional controller to stabilize the robot. A larger α means the robot falls faster and requires higher gains to recover.

To measure α, the motors were disabled and the robot was released from a small initial lean. The pitch angle was recorded as the robot fell freely. The linearized equation of motion for the pitch angle under free fall is:

$$\theta(t) = A \cdot \cosh(\lambda t) + B \cdot \sinh(\lambda t) \tag{4.1}$$

Where $\lambda^2 = \alpha$. Fitting this expression to the recorded pitch data gives λ directly, and squaring it gives α. Ten runs were conducted in total across two identification sessions. Runs where the initial pitch rate exceeded $15°/s$ were rejected, as they indicate that the robot was already moving at the moment of release. The weighed mean value across valid runs gave $\alpha = 67.5\,\mathrm{rad}^2/\mathrm{s}^2$. Session 1 produced a weighed mean of $64.5\,\mathrm{rad}^2/\mathrm{s}^2$ across four valid runs. Session 2 produced a weighed mean of $67.5\,\mathrm{rad}^2/\mathrm{s}^2$ across three valid runs. The session 2 result was adopted as the final value as it had lower overall uncertainty.

The free-fall fit for the dominant session 2 run is shown in Section C.2. It is important to note that these measurements were conducted prior to the hardware fault repairs described in Section 4.3. The free-fall test does not involve the encoders or the SPI bus, so the broken SS pin fault does not affect this measurement. However, the damaged encoder fault had not yet been identified, and while encoder data is not used in the free-fall test itself, it cannot be ruled out that other aspects of the hardware state differed from the post-fix configuration. The value $\alpha = 67.5\,\mathrm{rad}^2/\mathrm{s}^2$ is therefore reported as an estimate with the caveat that it was obtained on pre-fix hardware.

4.2.2 B_v Open-Loop Pulse Test

B_v quantifies how much wheel angular acceleration the motors produce per unit of commanded voltage. It appears in the B matrix of the state space model as the direct input to wheel velocity coupling and is needed to relate the controller output to the physical response of the drivetrain.

To measure B_v, the robot was placed on flat ground with the LQR controller disabled. An open-loop voltage pulse of 2V lasting 600 ms was applied to both motors simultaneously, and wheel angular acceleration was measured from encoder telemetry at 100 Hz. B_v was computed as:

$$B_v = \frac{V_{\text{end}} - V_{\text{start}}}{\text{duration} \cdot \text{Voltage}} \tag{4.2}$$

Five runs were conducted summarized in Section C.3. Runs 3 and 5 were rejected because the initial wheel velocity was high enough that the motors decelerated rather than accelerated during the pulse. The three valid runs gave individual values of 0.595, 0.541, and 0.837 $\mathrm{rad/s^2/V}$. Run 4 had a backward starting velocity that crossed zero during the pulse, which is valid but introduces more uncertainty than runs starting near rest. Taking the mean of the two cleanest runs (run 1 and 2) gives $0.568\,\mathrm{rad/s^2/V}$, while including run 4 gives $0.657\,\mathrm{rad/s^2/V}$. The final value of $B_v = 0.60\,\mathrm{rad/s^2/V}$ was adopted as a midpoint estimate.

This value was cross validated against the motor specification. The theoretical wheel angular acceleration per volt is:

$$B_v(\text{theoretical}) = \frac{2K_t}{Rrm} = 2 \cdot \frac{0.105}{11.1 \cdot 0.040 \cdot 0.753} = 0.629\,\mathrm{rad/s^2/V}$$

Where $K_t = 0.105\,\mathrm{Nm/A}$ is the motor torque constant derived from the GM4108H load data, $R = 11.1\,\Omega$ is the winding resistance, $r = 0.040\,\mathrm{m}$ is the wheel radius, and $m = 0.753\,\mathrm{kg}$ is the total mass. The theoretical value of $0.629\,\mathrm{rad/s^2/V}$ is within 5% of the measured value of $0.60\,\mathrm{rad/s^2/V}$, meaning that the measurement is in the correct range despite the hardware state uncertainty. This measurement is more sensitive to hardware state than the free fall test, as it depends directly on the encoder readings and SPI communication that were affected by the broken SS pin fault. The measurement was conducted prior to the hardware repairs, meaning the velocity readings could have been corrupted. Thus, the B_v value is reported with lower confidence than α and the motor spec cross-validation provides the primary basis for trusting the order of magnitude.

4.2.3 B_p Why It Cannot Be Measured

B_p quantifies how much angular acceleration the motors produce per unit of commanded voltage. It is the most critical parameter in the B matrix for LQR design, as it directly couples the control input to the unstable pitch dynamics that the controller must regulate. Unlike the previous two values, B_p cannot be measured directly on this platform. The fundamental difficulty is that any attempt to isolate pitch response to a motor input requires the robot body to be free to rotate while the wheelbase is held fixed. However, on a two-wheeled robot with no additional support structure, these two conditions cannot be achieved simultaneously. Three measurement approaches were attempted: holding the body while applying a voltage pulse, holding the wheel axle while applying a pulse, and allowing the robot to stand freely on the ground. All three produced unreliable and inconsistent results due to the coupling between wheel motion and body rotation that cannot be separated without additional instruments such as a dedicated test rig.

B_p can instead be derived analytically from the other identified parameters using the relationship:

$$B_p = \frac{\alpha r B_v}{g} \tag{4.3}$$

Where $r = 0.040\,\text{m}$ is the wheel radius and $g = 9.81\,\text{m/s}^2$. Substituting in the identified values gives $B_p = 0.172\,\text{rad/s}^2/\text{V}$. However, this derived value inherits uncertainty from both other values. Since B_v was measured on pre-fix hardware with potentially corrupted encoder readings, the derived B_p carries the same uncertainty and cannot be considered reliable.

4.2.4 Consequence for LQR Design

Since B_p cannot be measured directly and the derived estimate carries substantial uncertainty from pre-fix hardware measurements, the B matrix of the state-space model is incomplete. The Riccati equation requires both A and B to be fully known to compute the optimal gain matrix K analytically. With B_p unknown, this computation cannot be performed reliably on this platform. This is the underlying reason as for why the LQR methodology used in this thesis was used.

4.3 Hardware Fault Discovery

This section documents two hardware faults that were identified and resolved before valid data collection began. Both faults affected the reliability of the sensor feedback and had a direct impact on controller behavior during early testing.

4.3.1 Initial Observations

Prior to the identification of the hardware fault described in this section, a series of PID optimization runs and LQR hardware validation tests were conducted. In each session a consistent and unexplained behavioral anomaly was observed: the robot exhibited a persistent tendency to drift or accelerate in a single direction, particularly when subjected to external disturbances. This behavior was present regardless of the PID parameter set being tested and could not be resolved through tuning alone. The

LQR controller exhibited the same erratic behavior, with gains that appeared theoretically sound producing unstable or inconsistent responses on the physical hardware.

4.3.2 Broken SPI Slave Select Pin

The root cause of the observed issue was identified as a broken slave select (SS) pin on the SPI bus connecting the ESP32 microcontroller to the AS5048 wheel encoders. A broken SS pin causes the encoder to intermittently receive commands intended for other devices or fail to respond correctly, resulting in corrupted position readings being returned to the microcontroller. Wheel velocity is computed by differentiating encoder position over time. Corrupted position readings therefore produced incorrect velocity values, in some cases indicating the robot was moving at significant speed when it was stationary, or indicating zero velocity when it was accelerating. The velocity feedback provided to the PID velocity loop or the LQR velocity state was therefore unreliable throughout all pre-fix testing. The physical consequence was that the velocity loop was continuously correcting for velocity errors that did not exist, driving the robot in the direction of a ghost signal. This explains the consistent unidirectional runaway behavior observed in all earlier sessions. The fault was diagnosed by monitoring raw encoder telemetry and identifying implausible velocity readings that did not correspond to the robot's motion, followed by hardware inspection that confirmed the damaged pin.

4.3.3 Damaged Magnetic Encoder

Following the identification and replacement of the SS pin, a second hardware fault was identified. One of the two AS5048 magnetic encoders was producing intermittent velocity spikes with velocity values far exceeding the physical capability of the robot. Analysis of the logged trial data revealed individual samples reaching velocities of 41 rad/s or higher within otherwise normal measurement sequences. Since wheel velocity is averaged across the measurement window, these spikes inflated mean velocity scores significantly, producing artificially high objective function scores regardless of how well the robot was actually balancing. This fault was traced to physical damage on the encoder itself rather than the wiring or communication interface. The encoder was replaced before data collection continued.

4.3.4 Impact on the Study

Both hardware faults were identified and corrected before any data presented in this results chapter was collected. All optimization data from sessions prior to the hardware repairs was invalidated and excluded from the study. The faults affected both PID optimization and LQR hardware validation, meaning no reliable pre-fix performance benchmarks exist for either controller. A software level safeguard was also implemented in the optimization script following identification of the encoder fault. A corrupt telemetry threshold was added at 30.0 rad/s, a velocity that is physically impossible for this robot, which immediately terminates any trial in which such a reading is detected and assigns the maximum penalty score of 999. This prevents future encoder glitches from corrupting the optimization dataset even if the hardware degrades again during a session.

4.4 Objective Function Development

The development of a suitable objective function required four iterations before the optimizer could no longer exploit degenerate solutions. Each version was found to allow for a specific unintended balancing strategy that scored well without genuine upright balancing.

Version 1: The first iteration only measured peak-to-peak amplitude. The optimizer immediately exploited this by leaning the robot against the walls of the enclosure producing near-zero oscillation, satisfying the metric without the robot supporting its own weight. The function is the following:

$$\text{score} = \text{pitch}_{\text{max}} - \text{pitch}_{\text{min}}$$

Version 2: This iteration added a mean absolute pitch penalty weighed at twice the oscillation term. This penalized sustained lean but was insufficient when the lean angle was small, the robot could still achieve a low score by leaning slightly against the wall at a shallow angle. The function is the following:

$$\text{score} = \left(\text{pitch}_{\text{max}} - \text{pitch}_{\text{min}}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i|$$

Version 3: Added an inverse velocity penalty conditional on mean pitch exceeding 5°. The rationale was that a wall-leaning robot is stationary and penalizing stillness would discourage this strategy. The condition prevented penalizing a genuinely stable robot making small corrective movements. However, the optimizer found a new exploit — drifting freely across the enclosure produced high velocity, which was now rewarded. The function is the following:

$$\text{score} = \left(\text{pitch}_{\text{max}} - \text{pitch}_{\text{min}}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i| + \begin{cases} \frac{0.5}{|\bar{v}|+0.1} & \text{if } |\bar{\theta}| > 5° \\ 0 & \text{otherwise} \end{cases}$$

Version 4: The final formulation inverted the velocity logic by penalizing high velocity exponentially:

$$\text{score} = \left(\text{pitch}_{\text{max}} - \text{pitch}_{\text{min}}\right) + 2 \cdot \frac{1}{N} \sum_{i=1}^{N} |\theta_i| + e^{|\bar{v}|} - 1$$

The exponential function $e^x - 1$ is exactly zero at rest, grows slowly for small corrective movements, and grows aggressively for drifting. Two hard termination thresholds were added: 2.0 rad/s for runaway detection and 30.0 rad/s for corrupt telemetry detection. Both return a penalty score of 999 and exclude the candidate from future generations. The final formulation closes all known pathways, a low score is only achievable by a robot that is upright, not oscillating, and not drifting.

4.5 PID Optimization

This section presents the results of the cascade-PID optimization, conducted in two phases. Phase 1 performed a broad global search across the parameter space using four independent DE runs, and Phase 2 refined the search within a tightened region identified from the Phase 1 surrogate heatmaps.

4.5.1 Phase 1 Global Search

Four independent DE runs were conducted with identical bounds but different random seeds to explore the PID parameter space broadly. The initial Phase 1 bounds were:

Table 4.1.: Table 1: Initial Search Bounds

Parameter	Lower	Upper
stab_p	30.0	65.0
stab_i	40.0	100.0
stab_d	0.9	1.5
vel_p	0.005	0.020
vel_i	0.003	0.015

Results across all four Phase 1 runs are summarized in Table 4.2.

Table 4.2.: PID Phase 1 Summary

Run	Seed	Trials	Falls	Fall Rate	Best Score	stab_p	stab_i	stab_d	vel_p	vel_i
1	42	2040	5	0.2%	0.5018	64.00	63.13	1.485	0.0195	0.0095
2	67	2040	0	0.0%	0.8486	64.43	53.47	1.016	0.0185	0.0041
3	1337	2040	0	0.0%	2.8001	64.49	53.28	1.281	0.0187	0.0091
4	11	2040	0	0.0%	0.5407	64.77	49.11	1.277	0.0179	0.0030

The most significant finding from phase 1 was that stab_p converged to values of 64.0–64.8 across all four runs against an upper bound of 65.0. This consistent boundary contact indicated that the true optimum for stab_p lay above the initial search space. The optimizer was pushing against a wall in the stab_p dimension across all seeds, limiting the quality of solutions that could be found.

Run 3 with seed 1337 failed to find a good solution, achieving a best score of 2.8001, which was significantly worse than the other three runs. Examination of the convergence data revealed that this run initiated its population in a different region of the parameter space and never escaped the poor basin. This outcome illustrates the multi-basin nature of the PID parameter landscape and justifies the use of multiple seeds in phase 1.

Run 4 with seed 11 experienced a wall collision at trial 1590, producing a score of 693,084 and a mean velocity of 13.4 rad/s. This single event destabilized the population from generation 39 onward, resulting in a final best score of 0.5407 rather than the potentially lower value that the run was trending toward before the collision. Run 1 with seed 42 produced the best phase 1 score of 0.5018 and was selected as the global best candidate for phase 3. Surrogate landscape heatmaps generated after phase 1 confirmed the stab_p wall-pushing pattern, the dense clusters and best solution every run were positioned at the high stab_p corner of the bounds. The heatmaps also revealed that vel_p was consistently clustering near 0.018–0.020 across all runs, indicating this parameter was reliably identified within the phase 1 bounds. The stab_i and vel_i parameters showed more variation across runs, suggesting a flatter landscape in these dimensions.

4.5.2 Phase 2 Local Refinement

Phase 2 bounds were shifted upward for $stab_p$ based on the heatmap analysis, with other parameters tightened around the identified good region:

Table 4.3.: PID Shifted Search Bounds

Parameter	Lower	Upper
$stab_p$	55.0	85.0
$stab_i$	45.0	80.0
$stab_d$	1.0	1.6
vel_p	0.015	0.022
vel_i	0.003	0.012

One phase 2 run was conducted with seed 42, the result is summarized in Table 4.4.

Table 4.4.: PID Phase 2 Summary

Run	Seed	Trials	Falls	Best score	$stab_p$	$stab_i$	$stab_d$	vel_p	vel_i
Run 5	42	2040	0	0.0413	82.86	50.51	1.269	0.0197	0.0099

Run 5 achieved the best PID score of 0.0413, which is a 91.8% improvement over the best phase 1 score of 0.5018. The best trial had an amplitude of 0.027 degrees, a mean pitch of 0.005 degrees, and a mean velocity of 0.004 rad/s, indicating near-perfect upright balancing with minimal oscillation and virtually no drift. The convergence plot for run 5 showed a sharp drop to 0.041 by generation 16 followed by a stable plateau, with the mean absolute deviation from best $f(x)$ dropping simultaneously, the population was genuinely clustering around the best solution. Which can be seen in Fig. 4.3.

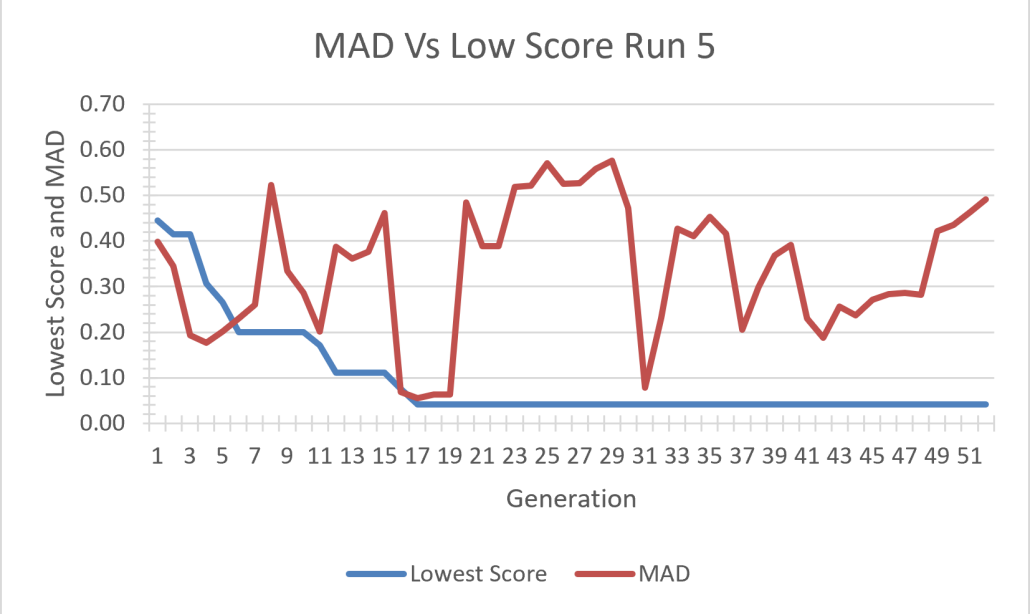


Figure 4.3.: Mean Absolute Deviation and Low Score

Run 5 showed a notable IMU drift mid-run where accumulated gyroscope integration error caused mean pitch readings to inflate to approximately 0.5–0.6 degrees across trials 512–1388. The pitch trim compensation partially mitigated this by subtracting the settle window mean from each measurement, but the population dynamics were still disrupted during the high-drift period. Despite this, run 5 still produced the best final score. Heatmap analysis of run 5 confirmed that vel_p and vel_i were the most tightly identified parameters, with excellent cluster tightness in the vel_p vs vel_i subplot. stab_d showed good identifiability. stab_i showed the weakest clustering across all parameter pairs, suggesting that the 3 second measurement window is insufficient for reliable identification of the integral gain (see Section B).

The final optimized PID parameters taken forward for comparison are:

Table 4.5.: Final PID Parameters

Parameter	Value
stab_p	82.86
stab_i	50.51
stab_d	1.269
vel_p	0.0197
vel_i	0.0099

4.6 LQR Optimization

This section presents the results of the LQR-form state-feedback optimization, following the same two-phase DE workflow applied to the cascade-PID controller. Results are reported for both phases and compared against the PID findings in Section 4.7.

4.6.1 Phase 1 Global Search

Four independent DE runs were conducted for LQR optimization using the same seeds as PID phase 1. The initial phase 1 bounds were:

Table 4.6.: Initial LQR Bounds

Parameter	Lower	Upper
k_1	60.0	110.0
k_2	2.0	6.0
k_3	1.0	3.0
k_4	0.005	0.15

Results for the completed phase 1 runs are summarized in Table 4.7.

Table 4.7.: LQR Phase 1 Summary

Run	Seed	Trials	Falls	Fall Rate	Best Score	k_1	k_2	k_3	k_4
Run 1	42	1682	113	6.7%	0.0476	84.96	5.55	1.012	0.083
Run 2	67	1682	0	0.0%	0.0359	98.31	4.51	1.154	0.072
Run 3	1337	1682	134	8%	0.0497	88.82	2.88	1.897	0.047
Run 4	11	1682	46	2.7%	0.0434	99.30	3.37	1.972	0.035

The LQR phase 1 runs produced notably different convergence behavior compared to PID phase 1, with all four seeds achieving competitive scores within phase 1 bounds alone. The best phase 1 score of 0.0359 from run 2 already surpasses the score that PID achieved during phase 2.

Run 2 with seed 67 produced the best result across the entire study at the time of phase 1 completion. It is the only run with zero falls across all 1682 valid trials, indicating that the LQR parameter space within the chosen bounds contain very few genuinely unstable combinations for seed 67's initial population. The convergence was smooth and improved steadily, the best $f(x)$ stepped down from 0.087 at generation 1 to 0.036 by generation 30, after which the curve plateaued for the remaining 20 generations. The MAD values stayed consistently low between 0.037 and 0.130, indicating genuine population clustering (see Fig. 4.4).

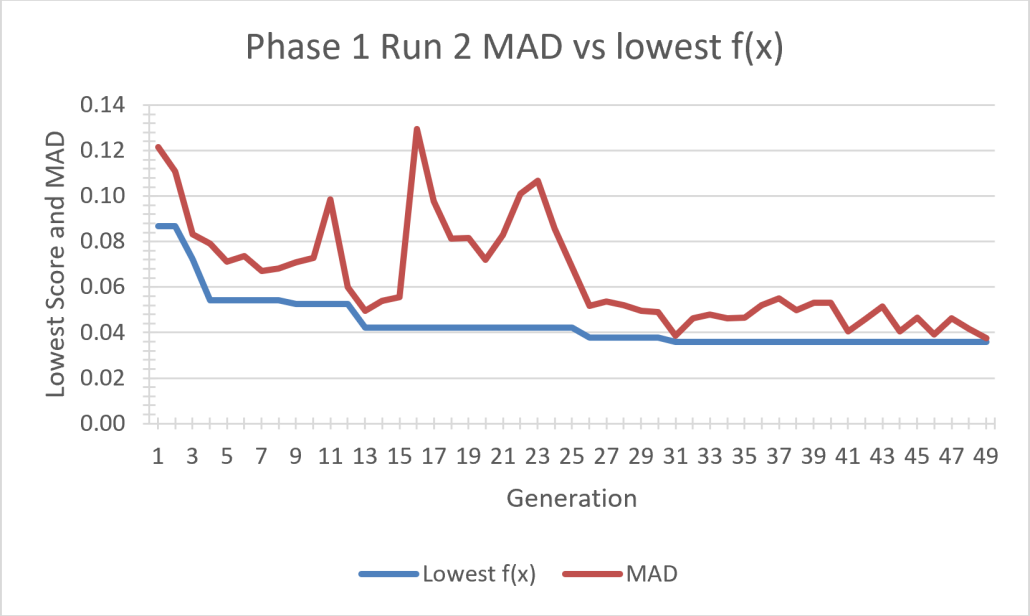


Figure 4.4.: Run 2 MAD vs Lowest f(x)

Run 1 with seed 42 recorded 113 falls and showed noisy convergence with consistently high MAD values of 0.3–1.1 caused by the fallen trials scoring 999. Despite this, the run converged to a competitive score of 0.0476, with the best $f(x)$ improving steadily from 0.218 to 0.048 across all 50 generations (see Fig. 4.5).

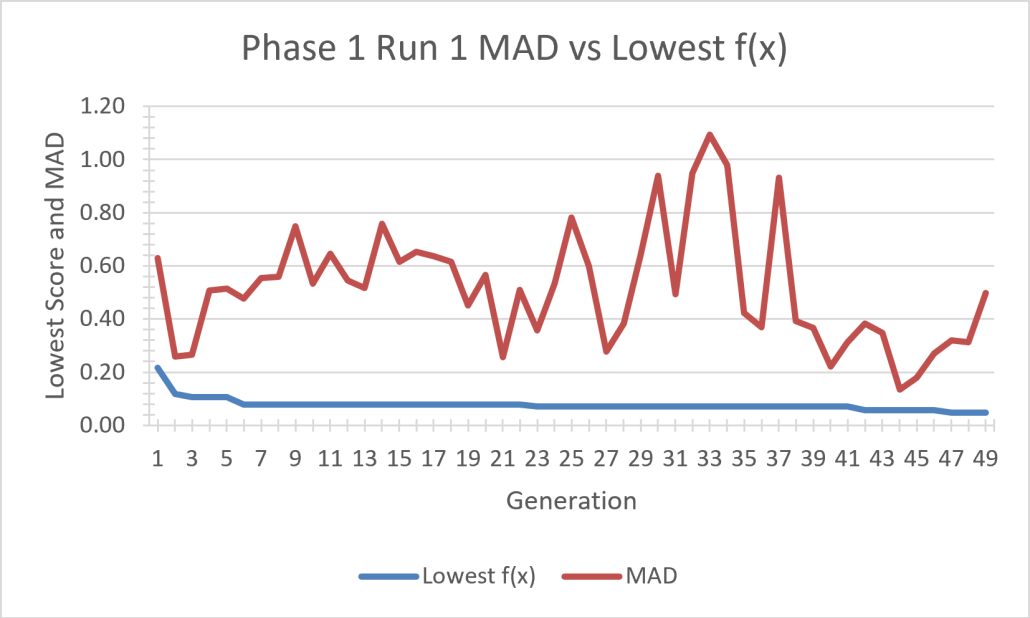


Figure 4.5.: Run 1 MAD vs Lowest f(x)

Run 3 with seed 1337 recorded the highest fall rate and found its best solution of 0.0487 at trial 139 in generation 4. Despite this early discovery, the best $f(x)$ remained completely flat for all 46 remaining generations, caused by the high fall rate. The MAD stayed consistently high at 0.4–1.3, confirming that the population never clustered successfully around the best solution (see Fig. 4.6).

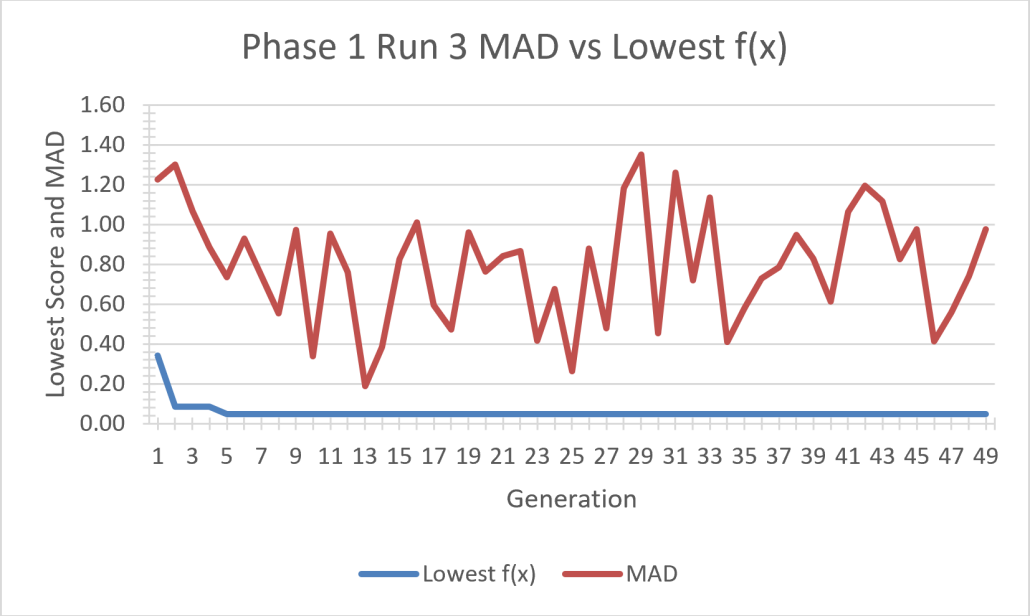


Figure 4.6.: Run 3 MAD vs Lowest f(x)

Run 4 with seed 11 produced the most interesting convergence pattern of the four runs. The fall rate was moderate at 2.7% and the MAD dropped to 0.04–0.07 from generation 27 onward, indicating genuine population clustering around a consistent good region. The best $f(x)$ improved steadily from 0.087 to 0.0485 by generation 24 and later found an even better solution with the score 0.0434 at generation 39. This indicates that the population exploited a local basin before a mutation event discovered a better neighbouring solution (see Fig. 4.7).

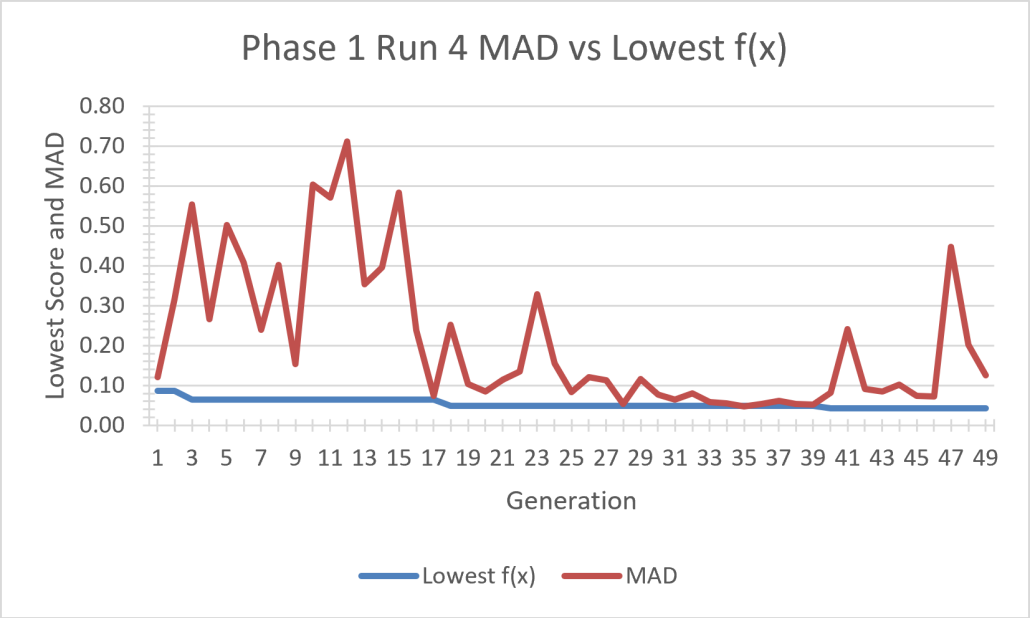


Figure 4.7.: Run 4 MAD vs Lowest f(x)

The fall rate variation across seeds is noteworthy. Unlike PID where falls were rare across all Phase 1 runs, LQR appears to have more regions of genuine instability within the parameter space. This is consistent with the fact that LQR gain selection has tighter stability constraints; gains that are too high, particularly in k_1 or k_2, can cause motor saturation and oscillation rather than simply producing a higher score as would be the case for suboptimal PID parameterization.

4.6.2 Phase 2 — Local Refinement

Phase 2 bounds were centered on the best solution from Phase 1 run 2 with the seed range shifted upward based on heatmap analysis showing clusters toward the upper end of the phase 1 bounds as visible in Section A. The following bounds were used as search bounds for phase 2:

Table 4.8.: LQR Shifted Bounds

Parameter	Lower	Upper
k_1	85.0	145.0
k_2	1.45	8.65
k_3	0.5	1.44
k_4	0.0	0.168

Run 5 achieved the best score of 0.0284, which is the best result across the entire study, surpassing both the best LQR phase 1 score of 0.0359 and the best PID phase 2 score of 0.0413. The best trial had an amplitude of 0.018 degrees, a mean pitch of 0.004 degrees, and a mean velocity of 0.005 rad/s. Zero falls were recorded across all 1632 trials, consistent with the phase 1 finding that the LQR parameter space contains few genuinely unstable combinations when operating near a good basin. The convergence pattern was clean and well-structured. Best $f(x)$ improved in two distinct steps, it started at 0.056 during generation 1 down to 0.041 by generation 7, holding stable for 12 generations, then dropping

sharply to 0.028 at generation 20. From generation 20 onward the best $f(x)$ remained completely flat for the remaining 30 generations. The MAD remained consistently low between 0.035 and 0.120 throughout as visible in Fig. 4.8, these are by far the lowest sustained MAD values in the entire study, confirming that the population clustered tightly around the good basin with no catastrophic events disrupting the dynamics.

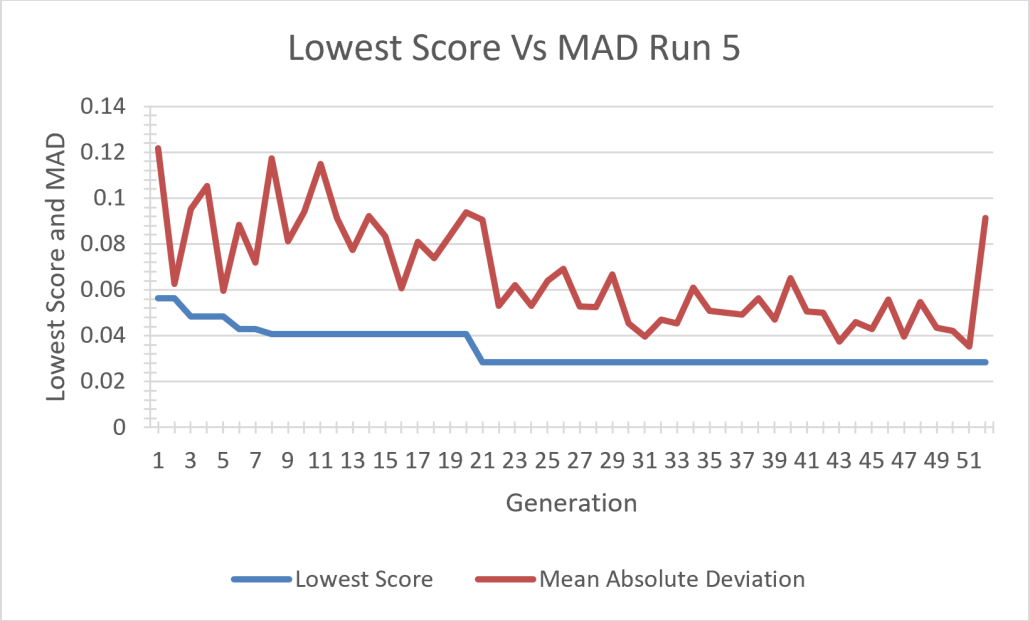


Figure 4.8.: Run 5 Convergence

The heatmap visible in Section A reveals a well converged landscape. The clusters sit comfortably inside the bounds, with no evidence of heavy wall pushing. The k_1 vs k_3 subplot shows the yellow region extending toward lower k_3 values, suggesting the optimal k_3 may sit slightly below the best trial value of 0.97. k_3 vs k_4 shows a tight cluster in a yellow green region. The overall heatmap structure is notably cleaner than any PID heatmap produced in the study.

The final optimized LQR parameters taken forward for comparison are:

Table 4.9.: LQR Final Values

Parameter	Value	Description
k_1	105.3	Pitch angle gain
k_2	5.05	Pitch rate gain
k_3	0.971	Velocity error gain
k_4	0.084	Velocity integral gain

4.7 Comparative Summary

This section brings together the final results from the PID and LQR optimization and provides a direct comparison of balancing performance and convergence behavior across both controllers.

4.7.1 Balancing Performance

The final optimized parameters and their associated performance metrics are summarized in Table 4.10. The PID results are taken from run 5 (Phase 2, seed 42) and the LQR results from run 5 (Phase 2, seed 67), as these produced the best composite scores across their respective optimization runs.

Table 4.10.: Final Controller Comparison

Metric	PID	LQR
Best competitive score	0.0413	0.0284
Pitch Amplitude	0.027°	0.018°
Mean Absolute Pitch	0.005°	0.004°
Mean Velocity	0.004 rad/s	0.005 rad/s
Parameters Optimized	5	4
Phase 1 best score	0.5018	0.0359
Phase 2 best score	0.0413	0.0284
Phase 1 fall rate range	0.0–0.2%	0.0–8.0%
Phase 2 fall rate	0.0%	0.0%
Total trials	10200	8160

The LQR-form controller achieved a composite score of 0.0284, which is 31% lower than the best PID score of 0.0413. On the individual metrics, LQR achieved 33% lower pitch amplitude and 20% lower mean absolute pitch. Mean velocity was marginally higher for LQR, indicating slightly more active wheel correction, but both values represent near-stationary operation well within the range of genuine upright balancing.

Both controllers achieved zero falls during their respective phase 2 runs, confirming that the final optimized parameter sets operate in stable regions of their respective parameter spaces. The qualitative difference in balancing behavior between the two controllers was also observable during testing. The LQR-form controller produced visibly smoother and more composed corrections, while the PID controller exhibited slightly more active oscillatory correction around the upright position.

4.7.2 Convergence behavior

Beyond final performance, the two controllers showed different convergence behavior throughout the optimization process. In phase 1, PID optimization produced inconsistent results across seeds. Three of four seeds found competitive scores between 0.50 and 0.85, but seed 1337 failed to escape a poor basin entirely, achieving a best score of 2.8001. All four seeds pushed stab_p against the upper bound of 65.0, indicating the initial search space did not contain the true optimum for this parameter. LQR phase 1 in contrast produced competitive scores across all four seeds, ranging from 0.0359 to 0.0497. No seed failed entirely, and the best phase 1 score of 0.0359 already surpassed the PID result after both phases. The heatmap still identified upward pressure on k_1, motivating a bound shift for phase 2, but the starting position was already strong. The mean absolute deviation tells a similar story. PID phase 2 showed disrupted population dynamics during the IMU drift period between trial 512 and 1388, with MAD values inflating during this window before recovering. LQR phase 2 produced the

cleanest convergence pattern in the entire study, with sustained MAD values of 0.035–0.120 throughout and a best $f(x)$ that stabilized completely from generation 20 onward with no disruptions.

4.7.3 Parameter space structure

The heatmap analysis across both optimization sessions reveals a structural difference between the two parameter spaces that is consistent with the convergence observations. The PID parameter space showed a steep landscape in the stab_p dimension, with the optimizer pushing hard against the upper bound. The stab_i dimension showed the weakest clustering of any parameter, with considerable variation across runs suggesting a flat landscape in that direction. The vel_p and vel_i parameters were the most reliably identified, with tight clusters visible in the vel_p versus vel_i subplot across multiple runs. The LQR parameter space showed a cleaner more uniform structure. All four phase 1 seeds found competitive solutions, the heatmap clusters sat comfortably inside the bounds with no heavy wall contact in phase 2, and the k_1 versus k_3 subplot showed a well-defined yellow region indicating a clear performance basin. The overall heatmap structure across both LQR phases was notably cleaner than any PID heatmap produced in the study. The structural difference between the two landscapes is consistent with a dimensionality and coupling argument: the five-gain cascade PID space has cross-parameter dependencies introduced by the coupling between the inner and outer loops, while the four-gain LQR-form space operates on a single unified control law with no such coupling. This structural property appears to make the LQR-form parameter space more uniformly searchable by differential evolution on this hardware.

5 Analysis

This chapter answers the three research questions by interpreting the optimization results presented in Section 4. Each section addresses one research question directly, drawing on the convergence data, heatmap analysis, and final performance metrics reported in Section 4.4–Section 4.7.

5.1 RQ1

Three design elements were necessary for DE to reliably find well-performing PID parameters. No single element was sufficient on its own; the failure of seed 1337 in Phase 1 demonstrates that the workflow alone cannot compensate for a poor basin, and the Phase 1 bound wall contact demonstrates that the objective function alone cannot compensate for an incorrectly specified search space.

First, the objective function required four iterations before it could no longer be exploited. Each version closed one specific degenerate solution: Version 1 was exploited by wall leaning, version 2 by shallow angle wall leaning, version 3 by free drifting, and version 4 closed all known pathways by penalizing velocity exponentially and adding harsh termination thresholds. A low score under version 4 is only achievable by a robot that is upright, not oscillating, and not drifting. The full development of the function is documented in Section 3.5 and the results of each version are reported in Section 4.4. This outcome is consistent with the theoretical properties of metaheuristic optimization described in Section 2.5.2, since Differential Evolution treats the objective function like a black box, evaluating $f(x)$ at candidate points without any knowledge of the intended solution [29]. When the objective function permits a low score because of a degenerate solution, the optimizer will find it not because it is misbehaving, but because it is doing exactly what the theory predicts. The iterative development of this function is therefore not a patch to a broken algorithm but a necessary consequence of applying a model-free optimizer to a physical system where the scoring function must be constructed rather than derived.

Second, a two-phase optimization workflow with surrogate-landscape feedback was required to identify the true optimum. Phase 1 revealed that stab_p was pushing against the upper bound of 65.0 across all four seeds, with the best solutions clustering at 64.0–64.77 (see Section B). Without the heatmap feedback loop, this would have gone undetected and the search would have ended in a suboptimal region. Shifting the bounds upward for phase 2 to 55.0–85.0 allowed stab_p to settle at 82.86, producing a final score of 0.0413.

Third, four independent seeds were necessary in phase 1. Run 3 with seed 1337 failed entirely, achieving a best score of 2.8001 while the other three seeds found scores below 0.85 (see Table 4.2). If only one seed had been used, there was a probability of selecting a run that initiated in a poor basin and never escaped. The multi-seed approach allowed the consistent bound-wall pattern to be confirmed across multiple seeds rather than being attributed to a single run's trajectory. This is well documented in evolutionary optimization literature, Das and Suganthan (2011) [33] note that DE's performance is sensitive to the initial population and that multiple independent runs are recommended to assess convergence reliability on non-convex problems.

5.1.1 Objective Function Convergence

The score magnitudes across phase 1 reveal how much the objective function constrained the achievable result even before the bound wall was corrected. All four runs produced best scores above 0.5, with generation mean scores regularly exceeding 1.0–3.0. These values are not evidence of poor balancing, rather they reflect version 4's exponential velocity penalty and hard thresholds set a much stricter definition of good balancing than earlier versions. The optimizer genuinely found the best solution available in the initial search space, but that search space was bounded on one side by the high stab_p ceiling. The order of magnitude drop from phase 1 to phase 2 scores is therefore attributable to the bound correction, not to an improvement in the objective function, confirming that the objective function was not the limiting factor. This is consistent with Awad et al. (2018) [34], who observe that the quality of the search space definition has a greater impact on final solution quality than the specific DE configuration used.

5.1.2 Phase 1 convergence analysis

The divergence between seed 1337 and the other three seeds demonstrates that basin dependent failure is a genuine risk in physical trial DE, not a theoretical concern. Three seeds found scores between 0.5 and 0.85 despite starting from different random positions (see Fig. 5.1), which means the performance landscape within the initial bounds has multiple accessible regions. Seed 1337 found none of them. This asymmetry, where three seeds succeed and one fails entirely, is precisely the outcome that a single-seed design cannot detect. With one seed, the study either gets lucky or it does not. With four seeds, the failure is visible and can be distinguished. The consistent stab_p bound wall contact across all four seeds is a separate and stronger signal: it appears regardless of whether the seed found a good basin or not, meaning it reflects a property of the search space rather than a property of any individual run. This is also consistent with a theoretical property of DE's mutation mechanism described in Section 2.6, where the difference vector $(x_b - x_c)$ that derives the mutation naturally scales with the spread of the population [31]. As the population converges against the stab_p boundary, the spread in that dimension collapses, mutation steps in that direction become small, and the optimizer stops exploring above the ceiling. Because of this the heatmap diagnostic is not just a visual check, it also detects the theoretical condition under which DE's self-scaling condition breaks down.

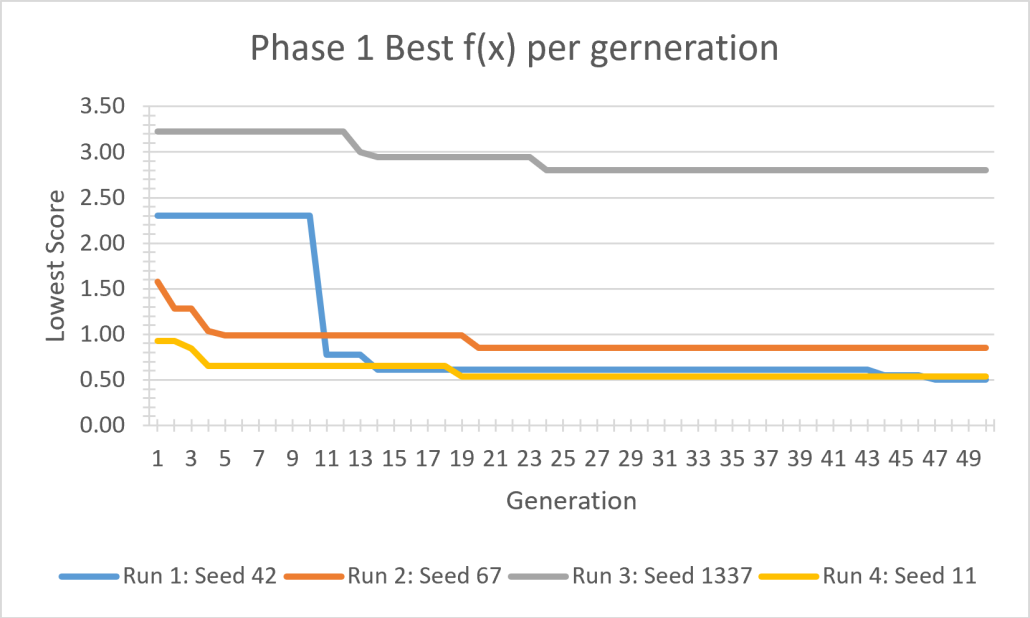


Figure 5.1.: Phase 1 best f(x) per generation

5.1.3 Phase 1 MAD analysis

The mean absolute deviation values across Phase 1 runs showed healthy population diversity throughout, with no run collapsing to near-zero MAD (see Fig. 5.2). This matters because a collapsed population is one that has effectively stopped searching, all candidates cluster around one point, and mutation steps become negligible. As described in Section 2.6, DE's selection mechanism ensures that the population never degrades, meaning each generation is at least as good as the previous one [32]. The absence of collapse across all four seeds indicates that the DE configuration (population size 8, 50 generations) maintained sufficient diversity even in a run like seed 1337 that never improved. The elevated MAD in seed 1337 is not a sign of a healthy search, it reflect a population that was exploring broadly because it never found a region worth converging around. High MAD can therefore indicate either productive early-stage exploration or persistent failure to identify a basin, and the best $f(x)$ trajectory is needed to distinguish the two.

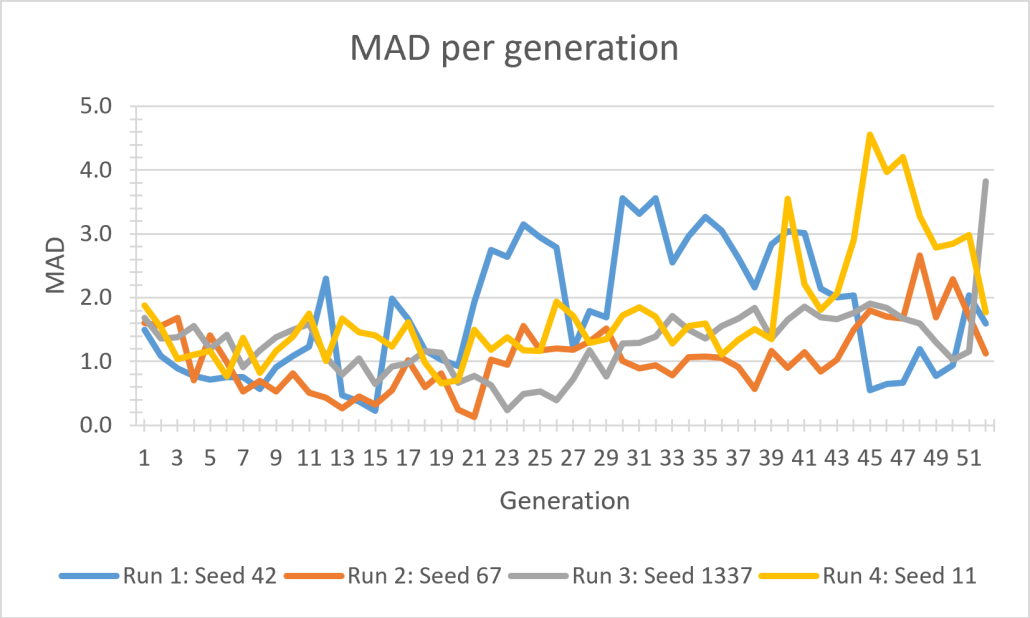


Figure 5.2.: Phase 1 MAD Per Generation

5.1.4 Phase 2 Convergence

The phase 2 run (seed 42, shifted bounds) showed a clear improvement trajectory from the phase 1 best of 0.5018 down to the final score of 0.0413 (see Fig. 5.3). This confirms that the bound correction was the critical intervention. The optimizer found the best accessible solution early on and then maintained it.

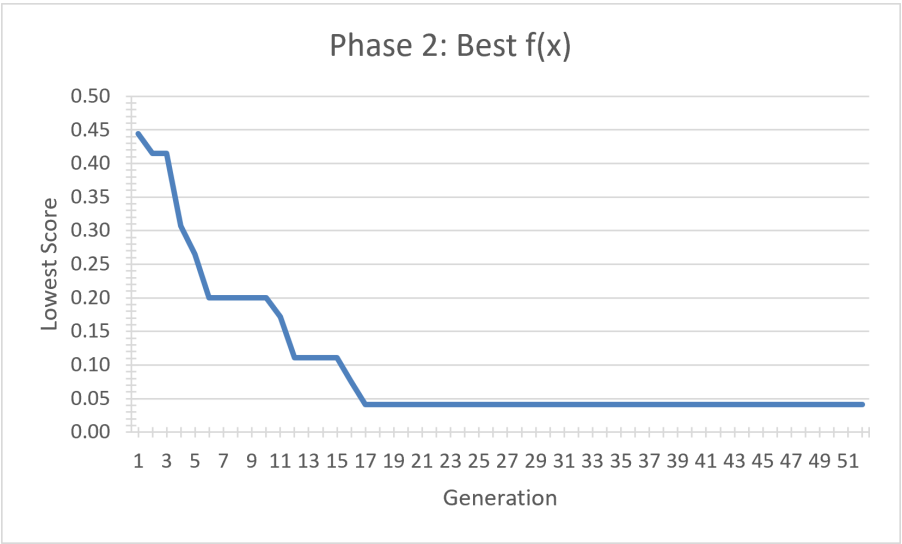


Figure 5.3.: Phase 2 Best f(x) per generation

The MAD values during phase 2 were disrupted during an IMU drift period visible in the pattern (see Fig. 5.4), however the best solution had already been found and was preserved by the greedy selection rule that prevents population degradation. This demonstrates an important property of DE: once a good solution enters the population it cannot be replaced by a worse one, making the best $f(x)$ trajectory robust to temporary measurement degradation even when the broader population is

disrupted [32]. This robustness to noisy evaluation is identified by Storn and Price (1997) [31] as one of DE's most practical advantages for engineering optimization and is directly observable in Phase 2 convergence data.

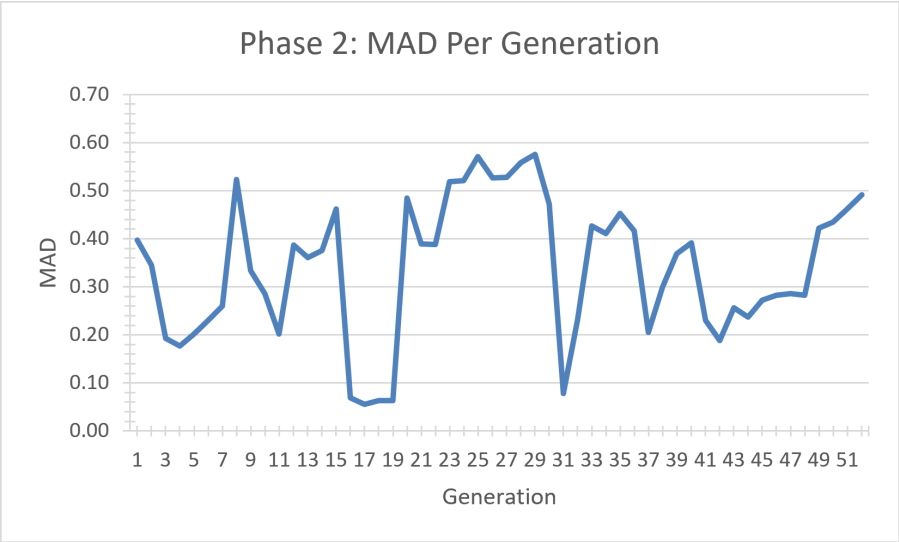


Figure 5.4.: Phase 2 MAD Per Generation

5.2 RQ2

The same objective function and workflow that were developed for PID also worked for LQR-form optimization, with no modifications required. All four phase 1 seeds produced competitive scores between 0.0359 and 0.0497 (see Table 4.7), in contrast to PID where one seed failed entirely. This indicates the LQR-form parameter space is more uniformly searchable within the initial bounds than the cascade-PID space. This is consistent with the properties of differential evolution described in Section 2.6.1, where it is described that DE performs better on smooth, unimodal landscapes rather than complex multi-modal ones, as the difference vector mutation explores more efficiently when there are fewer local optima to trap individual seeds [33]. The system identification in Section 4.2 confirms that the analytical path to an LQR solution was reachable in principle. α and B_v were measured, and B_p was derivable from them. However, since B_p is derived rather than measured directly, it inherits uncertainty from both α and B_v, the latter of the two being measured on pre-fix hardware with potentially corrupted encoder readings. It is important to note what this means in relation to the classical LQR formulation presented in Section 2.4. The Algebraic Riccati Equation minimizes the quadratic cost functional $J = \int (x^T Q x + u^T R u) dt$, producing a gain vector K that is analytically optimal with respect to that cost [10]. Without a reliable B matrix, the ARE cannot be solved and that optimality guarantee is absent. What this thesis uses is the LQR control structure $u = -Kx$ found empirically, essentially preserving the state-feedback architecture defined in Section 2.4 while not using the analytical optimality property. The DE-tuned gain vector $K = [105.3,\ 5.05,\ 0.971,\ 0.084]$ is therefore more trustworthy than solving the Riccati equation with an uncertain B_p, and the DE approach is not just a workaround but the more reliable tuning method available on this platform.

5.2.1 Phase 1 convergence

The most significant finding from LQR phase 1 is not which seed performed the best, but that all four performed competitively. Scores ranging from 0.0359 to 0.0487 (see Fig. 5.5) across four independent seeds indicate that the LQR-form parameter space contains accessible good regions from multiple starting positions, which is a property that the PID space did not demonstrate. This is consistent with the argument that a single unified control law produces a simpler landscape than a coupled cascade as described in Section 2.3 and Section 2.4. As noted in Section 2.3.1, the cascade PID structure creates cross-parameter dependencies because the velocity loop output sets the reference input for the stability loop, a change in vel_p changes the magnitude of the pitch reference signal, which in turn changes what stab_p needs to be to achieve a stable balance [9]. This interaction simply does not exist in the single-law LQR-form structure defined in Section 2.4, where $u = -Kx$ operates on all states through a single matrix multiplication. The optimizer does not need to navigate cross parameter dependencies between loops, so different starting positions are less likely to end up in irrecoverable basins.

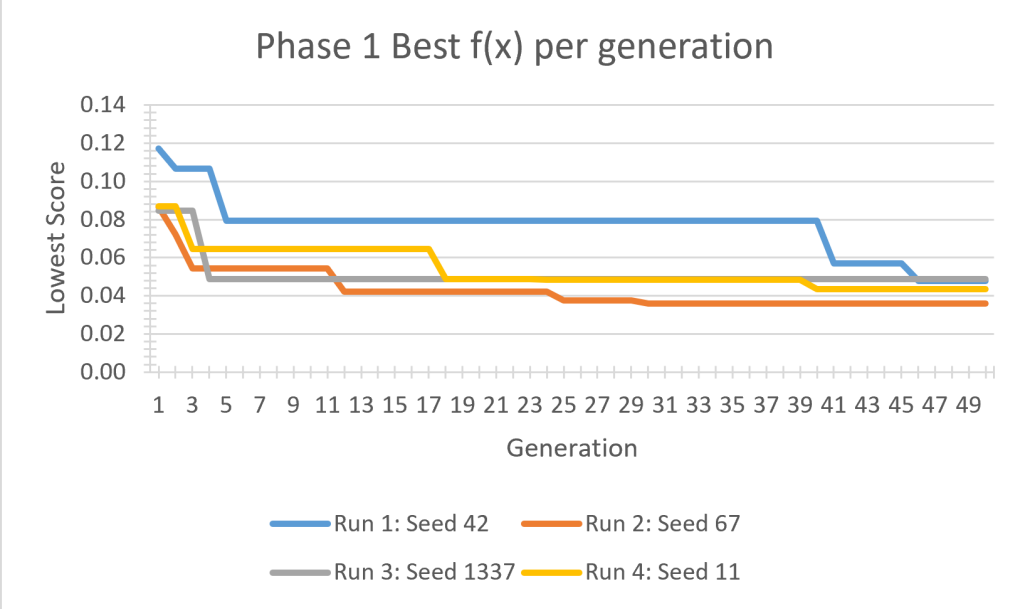


Figure 5.5.: Phase 1 Best f(x) per generation (LQR)

The fall rate variation across seeds reflects the cost of exploring a flat landscape. Unlike PID phase 1 where the bound on stab_p constrained exploration to a narrow strip, the LQR space has no such dominant constraint in phase 1. The optimizer therefore sampled more broadly, including regions near the stability boundary where high k_1 combined with low k_2 produces insufficient damping and falls. This is consistent with Section 2.3.1, where the linearized inverted pendulum characteristic equation requires sufficient dampening from k_2 to counteract the gravitational instability, so configurations with high k_1 but insufficient k_2 violate this condition and produce instability [4]. This is not a failure of the optimization, instead it is the expected behavior of DE exploring a flat landscape where the penalty gradient near the stability boundary is shallow, which is consistent with DE's documented behavior on underspecified problems [33]. The fact that all four seeds still converged to competitive scores despite these falls confirms that the objective function correctly penalized fallen trials without allowing them to contaminate the population.

5.2.2 Phase 2 convergence

The most significant feature of the run 5 convergence is the plateau between generation 7 and 20. The best $f(x)$ reached 0.0407 by generation 7 and held there for 12 generations before dropping to 0.0284 (see Fig. 5.6). This pattern indicates the population spent those 12 generations exploring around a local region before a mutation step found the better basin at generation 20. The fact that the improvement came late rather than early suggests the phase 2 landscape has a shallow gradient between the 0.04 and 0.03 score regions, requiring the population to sample broadly before the better region was discovered. Once found the best $f(x)$ never moved again, confirming that the population had reached a genuine optimum rather than a transitional plateau. This late stage improvement pattern is consistent with DE's documented behavior on smooth landscapes, Storn and Price (1997) [31] note that DE's population level exploration continues throughout the entire run, meaning improvements can occur at any generation rather than only early.

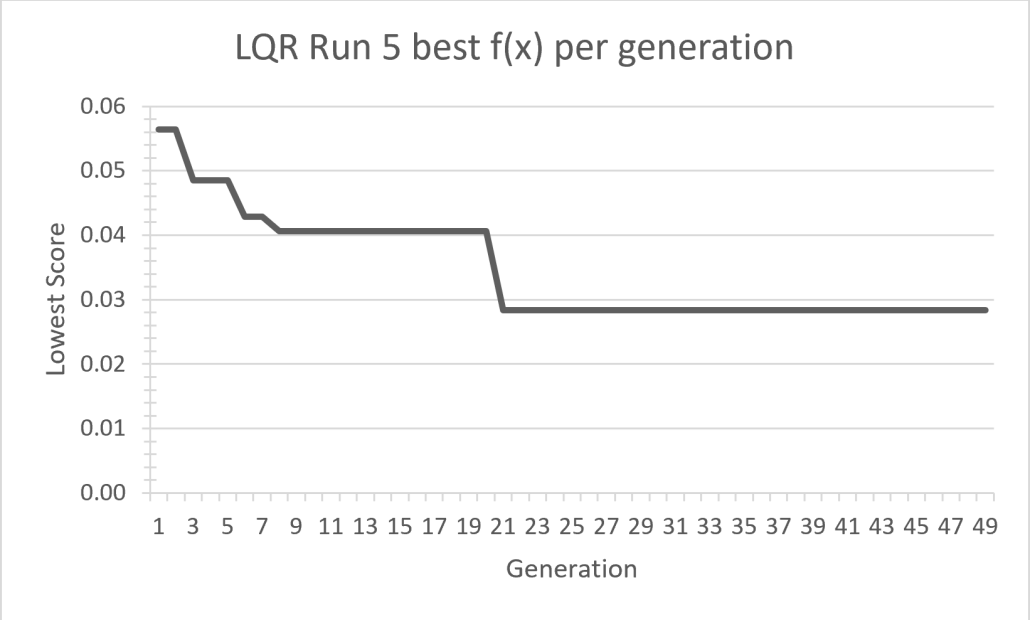


Figure 5.6.: LQR Phase 2 best f(x)

The consistent MAD range of 0.035–0.122 throughout all 50 generations (see Fig. 5.7), with no disruption comparable to the IMU drift episode in PID phase 2, reflects the smoother underlying landscape. In PID phase 2 the IMU drift inflated MAD to 0.389–0.576 because the population was operating near a sensitive region where small measurement errors produced large score changes. The LQR phase 2 MAD remaining tightly bound throughout suggests the population was operating in a region where the score is less sensitive to trial-to-trial noise, a property consistent with the broader, smoother basins visible in the Phase 2 heatmaps (see Section A).

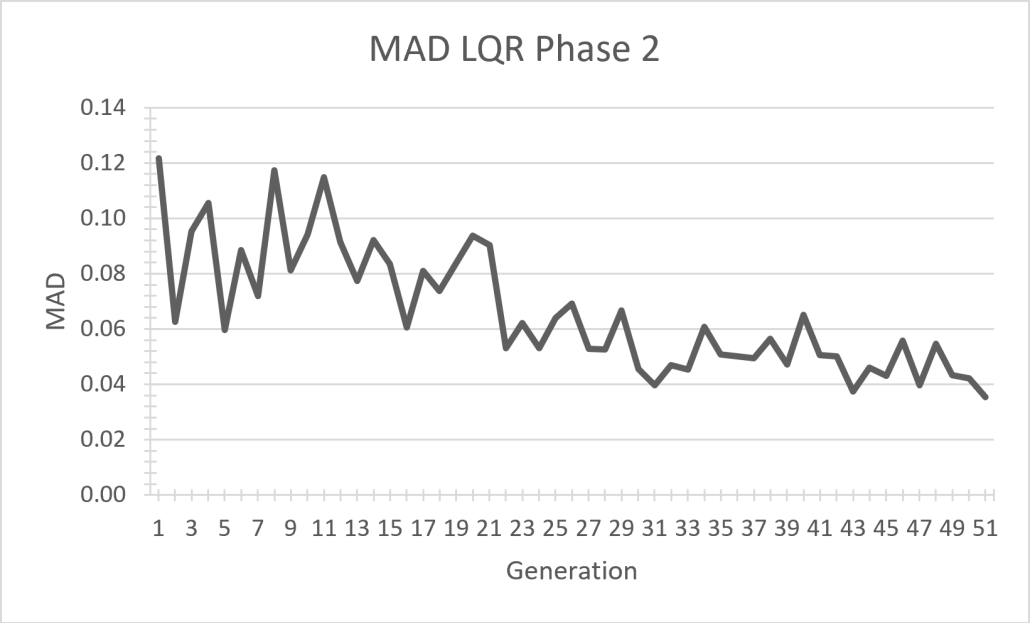


Figure 5.7.: MAD LQR Phase 2

5.3 RQ3

5.3.1 Balancing performance

The comparative results are summarized in Table 4.10 (Section 4.7). LQR-form achieved a composite score of 0.0284, which is 31% lower than the best PID score of 0.0413. Comparing individual metrics, LQR achieved 33% lower pitch amplitude (0.018° vs 0.027°) and 20% lower mean absolute pitch (0.004° vs 0.005°). Mean velocity was marginally higher for LQR at 0.005 rad/s compared to 0.004 rad/s for PID, indicating slightly more active wheel correction. Both controllers achieved zero falls during their respective phase 2 runs. This is consistent with the theoretical properties of the two control structures. As derived in Section 2.2.1, the linearized inverted pendulum dynamics have an unstable pole at $\pm\sqrt{\alpha}$ [4]. The LQR control law directly operates on θ and $\dot{\theta}$ as separate state variables, providing explicit dampening through k_2. The PID stability loop approximates dampening through the derivative term stab_d on pitch error, which is a less direct representation of the $\dot{\theta}$ state [9]. The theoretical framework therefore provides a basis for expecting the LQR-form to achieve tighter pitch regulation, which is consistent with the 33% lower amplitude observed.

5.3.2 Convergence Comparison

The convergence behavior of the two controllers differs in two important ways. First, the best LQR phase 1 score of 0.0359 already surpassed the best PID result of 0.0413 achieved after both phases of optimization. The PID workflow required bound correction and a full phase 2 run to reach 0.0413, while LQR reached a better score in phase 1 without any bound adjustment. This indicates that the LQR-form parameter space contains a more easily accessible performance basin with a reasonably chosen initial search region. Second, the population dynamics throughout LQR phase 2 were notably more stable than those observed in PID phase 2. The PID phase 2 run showed significant MAD

disruptions between generation 20 and 30, with values inflating to 0.389–0.576 due to IMU drift before recovering at generation 31. LQR phase 2 showed no such disruption, with MAD values remaining in a tight range of 0.035–0.122 throughout all 50 generations, and the best $f(x)$ showing a single clean improvement at generation 20 followed by complete stability.

5.3.3 Heatmap structure comparison

The most informative difference between the two optimization campaigns is visible in the heatmap structure. Across all four PID phase 1 runs, stab_p pushed consistently against its upper bound of 65.0, with the best solutions ranging from 64.00 to 64.77 (see Section B). The stab_i dimension showed the weakest clustering of any parameter in either campaign, the stab_i vs vel_i subplot in run 3 and run 4 shows an almost entirely uniform color field with no identifiable basin. Only vel_p showed consistently tight clustering across all four phase 1 PID runs. The LQR Phase 1 heatmaps show a different pattern (see Section A). Run 2 showed the most ordered landscape with a broad yellow region. Run 1 showed a well-defined horizontal band in the k_2 vs k_4 subplot and a tight k_3 range. Run 3 showed broad yellow regions in k_1 vs k_3 and k_3 vs k_4, indicating a flat but wide good region rather than a narrow basin. Run 4 showed a clear yellow band in the k_1 vs k_3 subplot indicating that higher k_1 with lower k_3 performs well, with k_4 showing leftward pressure toward its lower bound. No LQR phase 1 run showed the bound wall contact in a single dimension. The LQR phase 2 heatmap represents the cleanest landscape characterization in the entire study (see Section A). The "star-shaped" yellow region in k_1 vs k_2 and the tight basins in all remaining subplots confirm that phase 2 converged to a genuine interior optimum. The absence of bound-wall contact in any dimension, combined with the tightest MAD range of any run across both campaigns (0.035–0.122), provides strong evidence that the final LQR solution represents a well-characterized optimum. The structural difference between the two landscapes is consistent with both the theoretical formulations and the known limitations of each architecture. Section 2.3.1 identifies cross-parameter sensitivity as a fundamental limitation of PID controllers, noting that gain tuned for one operating point degrade when system parameters change [9], and the cascade structure compounds this by making the velocity loop the reference for the stability loop, creating parameter dependencies not existing in the LQR-form. Section 2.4 defines the LQR control law as a single matrix operation $u = -Kx$ with no nested loops and no inter-controller signal passing [10]. The optimization landscapes observed empirically, showing a complex and wall-prone one for PID and a smoother, more uniform one for LQR, are the practical manifestations of these theoretical structural differences on physical hardware. This finding is also consistent with the principle noted by Eiben and Smith (2015) [30] that population-based evolutionary algorithms perform better on lower-dimensional, less coupled search spaces, which is precisely the structural advantage the LQR-form parameterization provides.

6 Discussion

This chapter discusses the theoretical and practical implications of the findings reported in Section 4 and interpreted in Section 5, and identifies the methodological limitations that bound the conclusions of the study.

6.1 Theoretical Implications

This thesis set out to compare cascade-PID and LQR-form state feedback as control parameterizations under an identical DE-based tuning workflow. The result that the LQR-form converges to a better score is consistent with a structural argument: a single four-gain control law operating on a unified state vector presents a simpler optimization landscape than a five-gain cascade with coupled inner and outer loops. The cross-parameter dependencies in cascade-PID, where the velocity loop output sets the target pitch for the stability loop, create a more complex search space that is more sensitive to the starting region. This is an empirical result that is not obvious from theory alone, nothing in the formulation of either controller predicts that one would be more DE-tunable than the other on physical hardware. The objective function development in Section 4.4 carries a broader theoretical lesson. Each of the first three versions was exploited by the optimizer finding solutions that satisfied the mathematical definition of good balancing without actually balancing. As described in Section 2.5.2, metaheuristic methods treat the objective function as a black box, requiring only the ability to evaluate $f(x)$ at any point [29]. This is not a property of DE specifically but of any optimizer applied to an underspecified objective, since all the optimizer does is find the easiest path to the lowest possible score, not necessarily the intended solution. The four-iteration development process is a transferable methodology for any physical trial optimization problem where the objective function must be constructed from scratch. Any objective function that is nonzero on every degenerate solution will be exploited given sufficient trials. The two-phase workflow with surrogate-landscape feedback is a third transferable element. The heatmap-guided bound correction is not specific to this hardware or these controllers; it is a reusable diagnostic tool for any DE campaign where the initial search space may not contain the true optimum. As noted in Section 2.6.1, DE's difference vector mutation scales naturally with population spread [31], which means that when the population clusters against a boundary the algorithm effectively stops exploring. The pattern of bound wall contact detected across three of the four seeds is a reliable signal that cannot be attributed to chance, and the surrogate heatmap provides a principled basis for the correction rather than arbitrary bound expansion.

6.2 Practical implications

The primary practical finding is that LQR-form state-feedback is DE-tunable on hardware where system identification is incomplete. The conventional argument against LQR on platforms without accurate system models is that the Riccati equation cannot be solved, as the ARE defined in Section 2.4 requires a complete and reliable B matrix. This thesis demonstrates that the LQR control structure $u = -Kx$ can be retained and its gain vector found through physical trial optimization, bypassing the need for

analytical derivation. More importantly, the system identification results confirm that even when B_p can be derived, the inherited uncertainty from pre-fix hardware measurements makes the DE-tuned K the most trustworthy result. The DE approach is therefore not a fallback but the more reliable tuning method on this platform. The second practical implication is the value of comparing both controllers under identical conditions before claiming one is superior. Using the same objective function, the same workflow, and the same hardware for both campaigns means the performance difference is attributed to the control structure itself rather than the differences in tuning effort or measurement conditions. Future work comparing control structures on physical hardware should adopt the same principle. The third practical implication is the value of documenting the objective function iterations explicitly. The four version development in Section 4.4 is a concrete record of how a physical optimizer exploits an underspecified objective, and the fixes applied at each stage are directly reusable by any researcher applying DE to a similar balancing robot platform. Most published work on evolutionary controller tuning presents only the final objective function, where the failure modes that shaped it are usually hidden. Making them visible strengthens both reproducibility and the reader's ability to adapt the method on their own hardware.

6.3 Limitations

Several limitations bound the conclusions of this study.

The enclosure used as the test environment directly shaped the objective function development. Three out of the four objective function versions were driven by exploits that only exist because the robot was operating inside a confined space; for example, wall leaning requires a wall, and the enclosure dimensions limited the drift distance that could be penalized. A larger test environment may have produced different exploits requiring different fixes, meaning the objective function as developed is specific to this test setup and may require adaptation for other environments. The 3 second measurement window was insufficient for reliable identification of the integral gains. Heatmap analysis showed that stab_i in PID and k_4 in LQR both exhibited weaker clustering than the other parameters (see Section B and Section A), indicating that the optimization signal for slow acting integral terms is too weak to be reliably captured within the measurement window. A longer window would improve integral gain identifiability but increase trial duration and therefore total optimization time significantly. Both phase 2 optimizations were conducted with a single seed. While the phase 1 multi seed approach provides statistical confidence for the global search, the phase 2 result rests on a single optimization trajectory. Running phase 2 with multiple seeds would strengthen the claim that the final score represents a reliable optimum rather than a fortunate single seed result. B_p was derived rather than measured directly, inheriting uncertainty from both α and B_v. Since B_v was measured on pre-fix hardware with potentially corrupted encoder readings, the derived B_p carries the same uncertainty. The system identification results in Section 4.2 therefore cannot be used to validate the analytical LQR path with confidence, and the comparison between DE-tuned and Riccati-derived gains remains an open question for future work.

7 Conclusions/Concluding remarks

This chapter summarizes the conclusions drawn for each research question and identifies directions for future research.

7.1 Conclusions

RQ1: Reliable DE-based tuning on a physical self-balancing robot requires three jointly necessary elements: an objective function developed through four iterations to close all optimizer exploits, a two-phase workflow with surrogate heatmap feedback to detect and correct bound-wall contact across all seeds, and four independent seeds in phase 1 to guard against basin-dependent failure. Without all three elements the optimization either terminated in a suboptimal region or failed for individual seeds. The full development is documented in Section 3.5 and Section 4.4–Section 4.5.2, and the final optimized cascade-PID controller achieved a composite score of 0.0413 with a pitch amplitude of 0.027° and a mean velocity of 0.004 rad/s.

RQ2: The same objective function and two-phase workflow transferred directly to LQR-form optimization without modification. All four Phase 1 seeds produced competitive scores ranging from 0.0359 to 0.0487, in contrast to PID where seed 1337 achieved only 2.8001 and never escaped its initial basin, indicating a more uniformly searchable parameter landscape. As discussed in Section 5.2, the DE-tuned K preserves the state feedback structure defined in Section 2.4 while forgoing the optimality guarantee of the classical Riccati solution, which is a necessary consequence of B_p being unmeasurable on this platform. The system identification confirmed that the analytical path was reachable but that B_p inherits uncertainty, making the DE-tuned solution more trustworthy than a Riccati-derived alternative on this platform. The final optimized LQR-form controller achieved a composite score of 0.0284 with a pitch amplitude of 0.018° and a mean velocity of 0.005 rad/s, as documented in Section 4.6.2.

RQ3: LQR-form state-feedback outperformed cascade-PID on every primary balancing metric after equivalent optimization effort, achieving a 31% lower composite score, 33% lower amplitude, and a 20% lower mean pitch, as shown in Table 4.10 (Section 4.7). LQR-form demonstrated smoother convergence behavior, more stable population dynamics, and a cleaner surrogate landscape across all optimization runs. The best LQR phase 1 score of 0.0359 already surpassed the PID result of 0.0413 achieved after both phases, indicating the four-gain structure is more easily DE-tuned on this hardware than the five-gain cascade. This is consistent with the theory presented in Section 2.3 and Section 2.4, where the cascade architecture presents cross-parameter dependencies that the single-law LQR does not have, and these manifest empirically as a more complex and less accessible optimization landscape. The higher LQR phase 1 fall rates (0.0%–8.0% vs 0.0–0.2% for PID) are consistent with a flatter optimization landscape that causes the population to explore regions closer to the stability boundary before converging, rather than being constrained by a bound wall as in the PID campaign.

7.2 Future research

Three directions follow directly from the limitations of the study. First, measuring B_p directly on a test rig and estimating the moment of inertia of the pendulum through a swing test or CAD estimation would allow for a full Riccati solve as described in Section 2.4. This would enable a direct comparison between the analytically derived K^* and the DE-tuned $K = [105.3,\ 5.05,\ 0.971,\ 0.084]$. Whether the two vectors are close or diverge significantly would itself be a discussion worthy finding that could either validate the analytical model or reveal problems with unmodeled dynamics. Second, the generalizability of the tuned gains should be tested across different floor surfaces, load conditions, and disturbance magnitudes to characterize how well the near-zero-velocity optimization transfers to more demanding operating conditions. Third, the two-phase DE workflow could be extended to online adaptive tuning, where the robot retunes itself continuously during operation in response to changing conditions, removing the need for a separate offline tuning phase entirely.

References

[1] T. Braunl, *Embedded robotics: From mobile robots to autonomous vehicles with raspberry pi and arduino*, 4th ed. Springer, 2022. doi: 10.1007/978-981-16-0804-9.

[2] A. Wang, "What is a mobile robot? Definition, types, and applications explained." Accessed: Feb. 16, 2026. [Online]. Available: https://www.fdatabot.com/what-is-a-mobile-robot-definition-types-and-applications-explained/

[3] A. Mathew, R. Ananthu, P. Binsy, A. Vahid, C. Thomas, and S. Sidharthan, "Design and control of a two-wheel self-balancing robot," *IOP Conference Series: Materials Science and Engineering*, vol. 1114, no. 1, p. 012058, 2021, doi: 10.1088/1757-899X/1114/1/012058.

[4] K. M. Goher, "A two-wheeled machine with a handling mechanism in two different directions," *Robotics and Biomimetics*, vol. 3, no. 1, p. 17, 2016, doi: 10.1186/s40638-016-0049-8.

[5] S. E. Arefin, "Simple two-wheel self-balancing robot implementation." 2023. doi: 10.48550/arxiv.2303.12067.

[6] Fiveable Content Team, "Pivot point."

[7] Y. Olmez, G. O. Koca, and Z. H. Akpolat, "Clonal selection algorithm based control for two-wheeled self-balancing mobile robot," *Simulation Modelling Practice and Theory*, vol. 118, p. 102552, 2022, doi: 10.1016/j.simpat.2022.102552.

[8] F.-C. Paulescu, I. Szeidert, I. Filip, and C. Vasar, "Two-wheeled self-balancing robot," 2021.

[9] O. A. Somefun, K. Akingbade, and F. Dahunsi, "The dilemma of PID tuning," *Annual Reviews in Control*, 2021, doi: 10.1016/j.arcontrol.2021.05.002.

[10] S. J. Chacko, P. C. Nair, and R. J. Abraham, "Optimizing LQR controllers: A comparative study," *Results in Control and Optimization*, vol. 14, p. 100387, 2024, doi: 10.1016/j.rico.2024.100387.

[11] N. Phong-Luu *et al.*, "PID controller for balancing one-wheeled self-balancing robot," *Roboica şi Management*, vol. 27, no. 1, pp. 23–27, 2022, doi: 10.24193/rm.2022.1.5.

[12] Cambridge University Press, "Nonlinear." [Online]. Available: https://dictionary.cambridge.org

[13] K. H. Lundberg and T. W. Barton, "History of inverted-pendulum systems," *IFAC Proceedings Volumes*, vol. 42, no. 24, pp. 131–135, 2010, doi: 10.3182/20091021-3-JP-2009.00025.

[14] O. Boubaker, "The inverted pendulum: A fundamental benchmark in control theory and robotics," in *International conference on education and e-learning innovations*, 2012. doi: 10.1109/ICEELI.2012.6360606.

[15] S. Kim and S. Kim, "On the dynamic model of a two-wheeled inverted pendulum robot." 2014.

[16] W. Hu, "PID controller," 2025, Available: https://ace.ewapub.com/article/view/22912.pdf

[17] E. Çelik, N. Öztürk, Y. Arya, and C. Ocak, "(1+PD)-PID cascade controller design for performance betterment of load frequency control in diverse electric power systems," *Neural Computing & Applications*, vol. 33, no. 22, pp. 15433–15456, 2021, doi: 10.1007/s00521-021-06168-3.

[18] T. R. Kuphaldt, "Cascade control." [Online]. Available: https://control.com/textbook/basic-process-control-strategies/cascade-control/

[19] P. Burggraef, A. R. Pérez Martínez, H. Roth, and J. Wagner, "Quadrotors in factory applications: Design and implementation of the quadrotor's P-PID cascade control system," *SN Applied Sciences*, vol. 1, no. 7, 2019, doi: 10.1007/s42452-019-0698-7.

[20] F. Jiao, H. Qiao, K. Li, X. Tong, and R. Zhu, "Improved adaptive cascade predictive control for trajectory tracking of a crawler hydraulic drill-anchor robot with slippage compensation," *Machines*, vol. 14, no. 2, p. 230, 2026, doi: 10.3390/machines14020230.

[21] L. Guo, S. A. A. Rizvi, and Z. Lin, "Optimal control of a two-wheeled self-balancing robot by reinforcement learning," *International Journal of Robust and Nonlinear Control*, vol. 31, no. 6, pp. 1885–1904, 2021, doi: 10.1002/rnc.5058.

[22] Z. R. Wani *et al.*, "A critical review on control strategies for structural vibration control," *Annual Reviews in Control*, vol. 54, pp. 103–124, 2022, doi: 10.1016/j.arcontrol.2022.09.002.

[23] Z. Yuan, Z. Zhang, L. Zeng, and X. Li, "Key technologies in active microvibration isolation systems: Modeling, sensing, actuation, and control algorithms," *Measurement*, vol. 222, p. 113658, 2023, doi: 10.1016/j.measurement.2023.113658.

[24] G. R. Gonçalves da Silva, A. S. Bazanella, C. Lorenzini, and L. Campestrini, "Data-driven LQR control design," *IEEE Control Systems Letters*, vol. 3, no. 1, pp. 180–185, 2019, doi: 10.1109/LCSYS.2018.2868183.

[25] N. S. Mehta, V. Bhaiya, K. A. Patel, and S. Elias, "Optimal design of linear quadratic regulator controller for asymmetric structure using metaheuristic algorithm," *The Structural Design of Tall and Special Buildings*, vol. 35, no. 1, 2026, doi: 10.1002/tal.70128.

[26] Yu. K. Taranenko, O. Yu. Oliinyk, and V. V. Lopatin, "Using LQR controller for optimizing UAV control systems," *Cybernetics and Systems Analysis*, vol. 62, no. 1, pp. 152–164, 2026, doi: 10.1007/s10559-026-00854-7.

[27] M. Waszak and R. Langowski, "An automatic self-tuning control system design for an inverted pendulum," *IEEE Access*, vol. 8, pp. 26726–26738, 2020, doi: 10.1109/ACCESS.2020.2971788.

[28] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. Springer, 2006. doi: 10.1007/978-0-387-40065-5.

[29] E.-G. Talbi, *Metaheuristics: From design to implementation*. Wiley, 2009. doi: 10.1002/9780470496916.

[30] A. E. Eiben and J. E. Smith, *Introduction to evolutionary computing*, 2nd ed. Springer, 2015. doi: 10.1007/978-3-662-44874-8.

[31] R. Storn and K. Price, "Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997, doi: 10.1023/A:1008202821328.

[32] V. Feoktistov, *Differential evolution: In search of solutions*. Springer, 2006.

[33] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011, doi: 10.1109/TEVC.2010.2059031.

[34] N. H. Awad, M. Z. Ali, R. Mallipeddi, and P. N. Suganthan, "An improved differential evolution algorithm using efficient adapted surrogate model for numerical optimization," *Information Sciences*, vol. 451–452, pp. 326–347, 2018, doi: 10.1016/j.ins.2018.04.024.

[35] W. L. Woo and B. Gao, *Sensor signal and information processing III*. MDPI, 2021.

[36] T. Musha and H. Inujima, "The basic examination of the magnetic rotation angle sensor with the magnetic substance slit," *IEEJ Transactions on Fundamentals and Materials*, vol. 140, no. 6, pp. 306–311, 2020, doi: 10.1541/ieejfms.140.306.

[37] B. Schabron, A. Reust, J. Desai, and Y. Yihun, "Integration of forearm sEMG signals with IMU sensors for trajectory planning and control of assistive robotic arm," 2019.

[38] M. Quante, E. R. Kaplan, M. Rueschman, M. Cailler, O. M. Buxton, and S. Redline, "Practical considerations in using accelerometers to assess physical activity, sedentary behavior, and sleep," *Sleep Health*, vol. 1, no. 4, pp. 275–284, 2015, doi: 10.1016/j.sleh.2015.09.002.

[39] Robolink, "3.4: gyroscope." [Online]. Available: https://learn.robolink.com

[40] S. Campbell, "Basics of the I2C communication protocol." [Online]. Available: https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol

[41] ams AG, "AS5048A/AS5048B product document," ams OSRAM Group, 2018.

[42] Asahi Kasei Microdevices, "#05 principle and advantages of magnetic encoder." [Online]. Available: https://www.akm.com

[43] N. Bong, "What is an actuator?" [Online]. Available: https://www.progressiveautomations.com

[44] Monolithic Power Systems, "Brushless vs. Brushed DC motors: When and why to choose one over the other." [Online]. Available: https://www.monolithicpower.com/en/learning/resources/brushless-vs-brushed-dc-motors

[45] MOONS Industries, "Brushless motor driving principle." [Online]. Available: https://www.moonsindustries.com/article/brushless-motor-driving-principle

[46] TLC, "Understanding stators & rotors: Key components of electric motors." [Online]. Available: https://www.tlclam.net/understanding-stators-rotars-key-components-electric-motors/

[47] C. Hashemi-Pour, "Definition: What is a microcontroller (MCU)?" TechTarget, 2024.

[48] A. Fallatah, "The versatile world of arduino boards: A comprehensive review," *Juniper Online Journal Material Science*, vol. 9, no. 1, 2024, doi: 10.19080/jojms.2024.09.555753.

[49] SimpleFOC Project, "Simple FOC project." [Online]. Available: https://simplefoc.com/

[50] Handson Technology, "WeMOS D1 R32 ESP32 Wi-Fi and bluetooth board." [Online]. Available: https://handsontec.com/dataspecs/module/ESP/WeMos%20D1%20R32.pdf

[51] Wikipedia, "ESP32." [Online]. Available: https://en.wikipedia.org/wiki/ESP32

[52] R. Keote, A. Ramteke, R. Choudhari, P. Pusadekar, P. Hande, and N. Chouragade, "Design & implementation of self balancing bot," in *International conference on sentiment analysis and deep learning*, 2024.

[53] M. M. I. Muhiuddin, A. A. Taki, K. Zahin, M. J. Karim, and S. M. Al Kais, "Optimized control strategy for self-balancing robot: Micro-stepping motors and PID algorithm," in *International conference on robotics, automation, artificial-intelligence and internet-of-things*, Dhaka, Bangladesh, 2024.

[54] A. Skuric, H. S. Bank, R. Unger, O. Williams, and D. González-Reyes, "SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors," *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022, doi: 10.21105/joss.04232.

[55] A. Ganeshpurkar, V. Pandey, S. Asati, R. Maheshwari, M. Tekade, and R. K. Tekade, "Experimental design and analysis of variance," in *Dosage form design considerations*, vol. 2, Academic Press, 2018. doi: 10.1016/B978-0-12-814421-3.00008-7.

[56] P. R. Jain, U. S. Khan, and P. R. Jain, "Research designs and methodologies," 2025. doi: 10.1007/978-981-96-5154-2_2.

[57] L. C. Segura, A. L. F. d. S. Vasconcelos, V. G. Slomski, and S. Subrahmanyam, "Development of research design," 2025. doi: 10.4018/979-8-3373-4432-4.ch002.

[58] S. Bell, R. Kitchin, and N. Thrift, "Experimental design," in *International encyclopedia of human geography*, Elsevier, 2009, pp. 672–675. doi: 10.1016/B978-008044910-4.00431-4.

[59] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[60] Adafruit Industries, "Adafruit TDK InvenSense ICM-20948 9-DoF IMU (MPU-9250 upgrade) — STEMMA QT / Qwiic." [Online]. Available: https://www.adafruit.com/product/4554

[61] P. Slater and F. Hasson, "Quantitative data quality assurance, analysis and presentation," *Journal of Psychiatric and Mental Health Nursing*, vol. 32, no. 3, pp. 723–727, 2025, doi: 10.1111/jpm.13143.

[62] J. M. Frambach, C. P. M. van der Vleuten, and S. J. Durning, "AM last page: Quality criteria in qualitative and quantitative research," *Academic Medicine*, vol. 88, no. 4, pp. 552–552, 2013, doi: 10.1097/ACM.0b013e31828abf7f.

[63] I. Ahmed and S. Ishtiaq, "Reliability and validity: Importance in medical research," *Journal of the Pakistan Medical Association*, vol. 71, no. 10, pp. 2401–2406, 2021, doi: 10.47391/JPMA.06-861.

[64] H. Noble and J. Smith, "Ensuring validity and reliability in qualitative research," *Evidence-Based Nursing*, vol. 28, no. 4, pp. 206–208, 2025, doi: 10.1136/ebnurs-2024-104232.

[65] D. F. Polit and C. T. Beck, "Generalization in quantitative and qualitative research: Myths and strategies," *International Journal of Nursing Studies*, vol. 47, no. 11, pp. 1451–1458, 2010, doi: 10.1016/j.ijnurstu.2010.06.004.

[66] J. Reiss and J. Sprenger, "Scientific objectivity," in *The stanford encyclopedia of philosophy*, E. N. Zalta, Ed., Metaphysics Research Lab, Stanford University, 2020.

[67] N. van Dongen and M. Sikorski, "Objectivity for the research worker," *European Journal for Philosophy of Science*, vol. 11, no. 3, p. 93, 2021, doi: 10.1007/s13194-021-00400-6.

A LQR Heatmaps

A.1 LQR Run 1 Phase 1

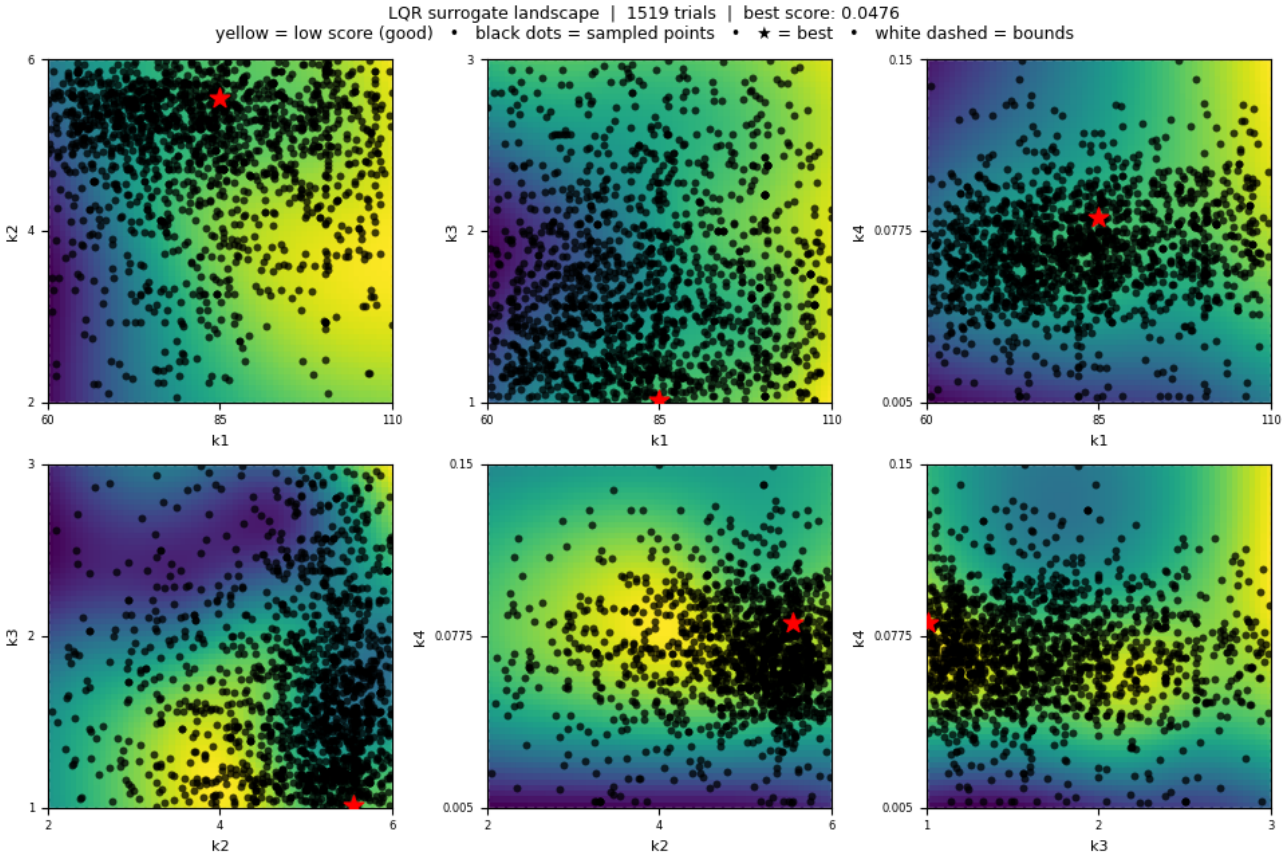


Figure A.1.: LQR Run 1 Phase 1 Heatmap

A.2 LQR Run 2 Phase 1

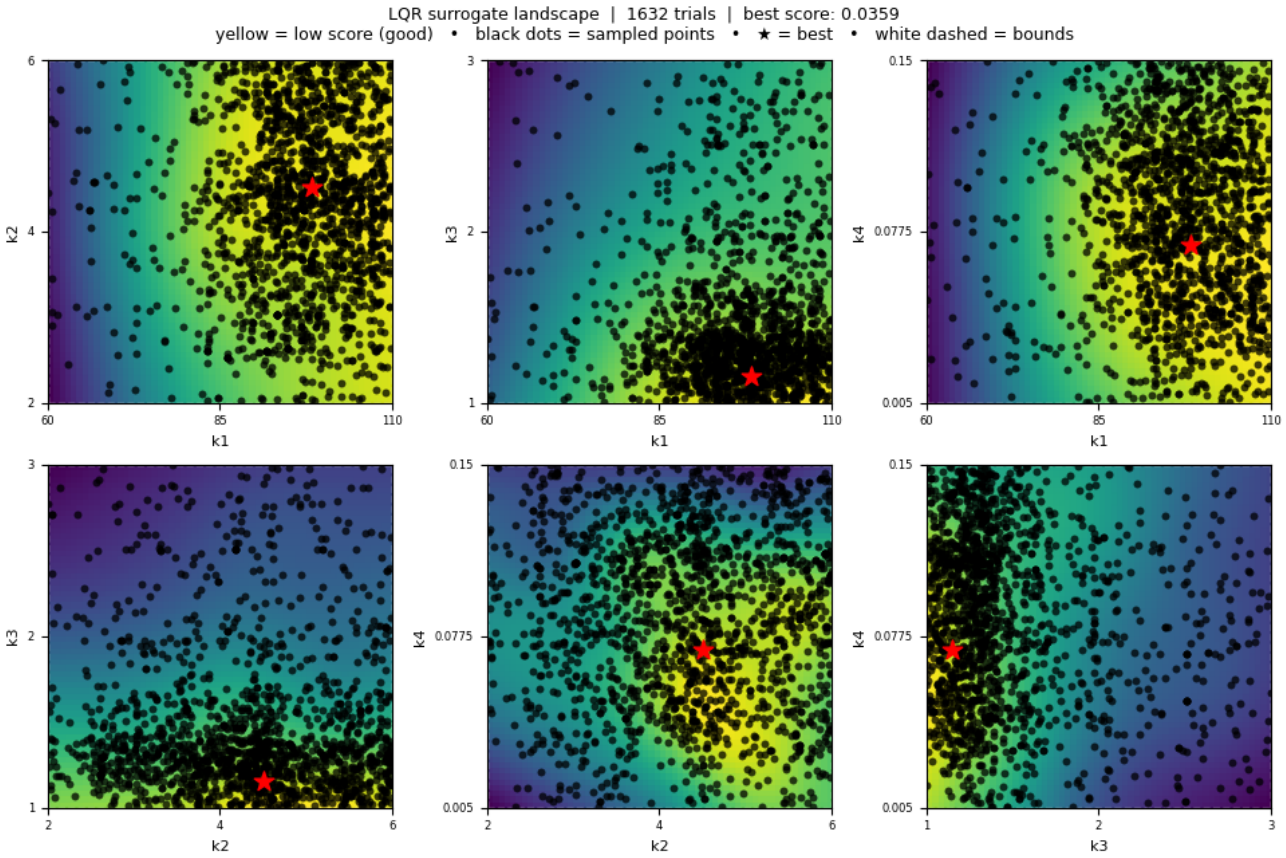


Figure A.2.: LQR Run 2 Phase 1 Heatmap

A.3 LQR Run 3 Phase 1

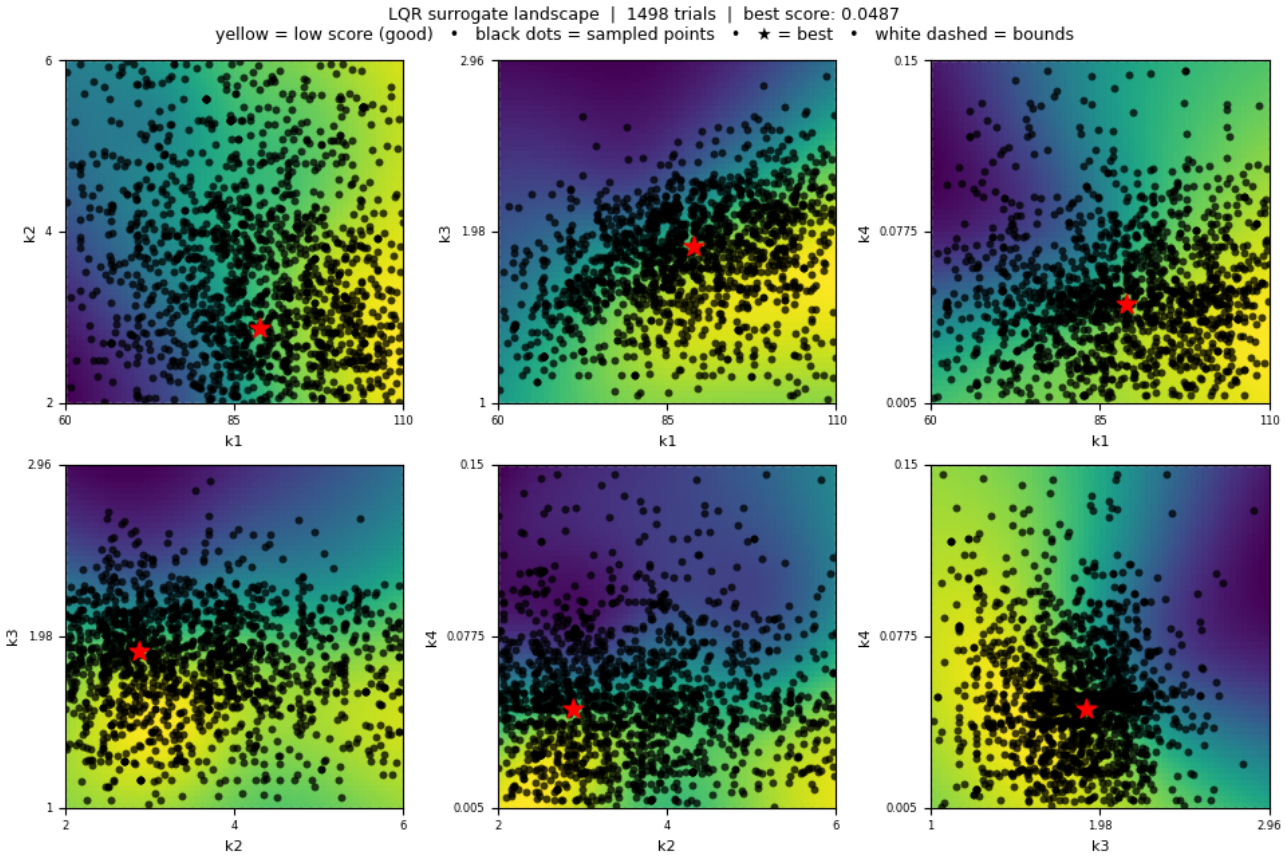


Figure A.3.: LQR Run 3 Phase 1 Heatmap

A.4 LQR Run 4 Phase 1

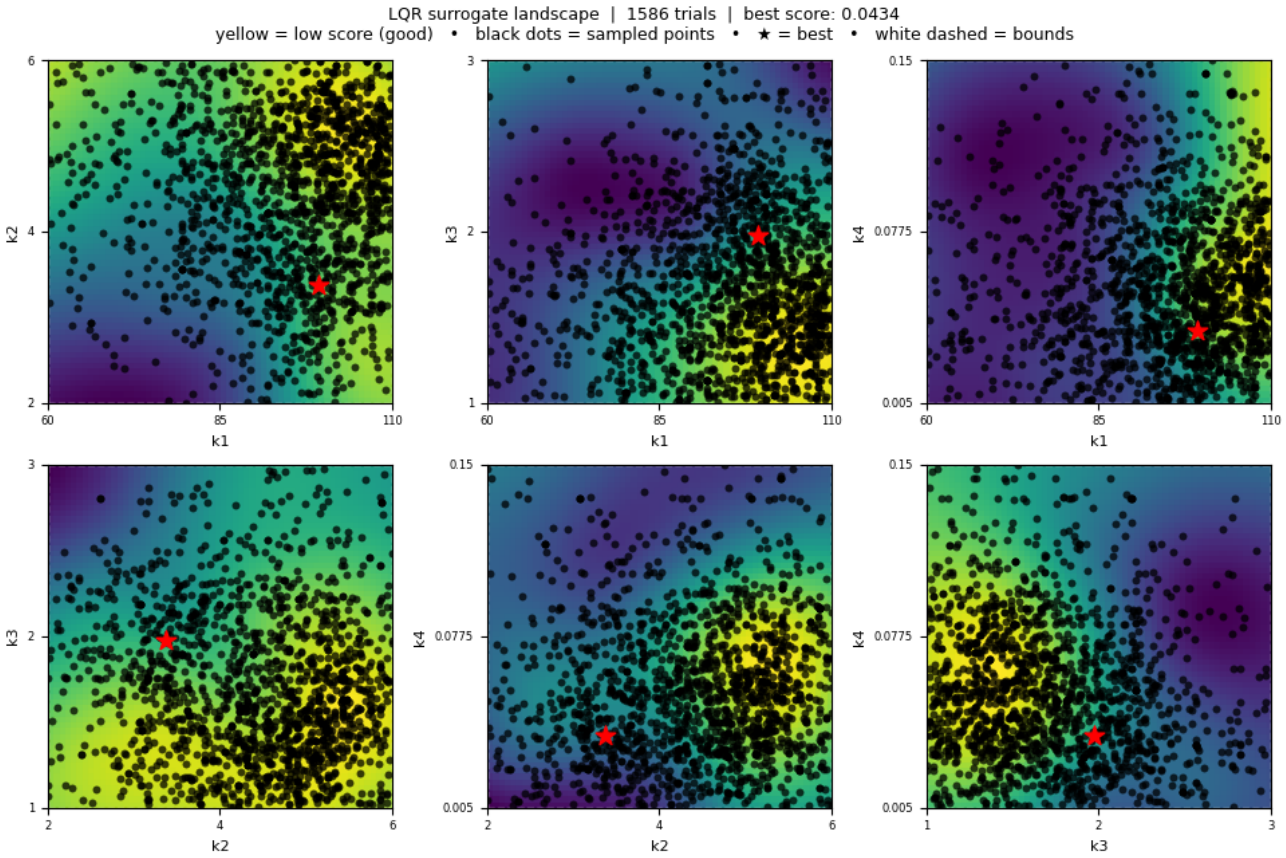


Figure A.4.: LQR Run 4 Phase 1 Heatmap

A.5 LQR Run 5 Phase 2

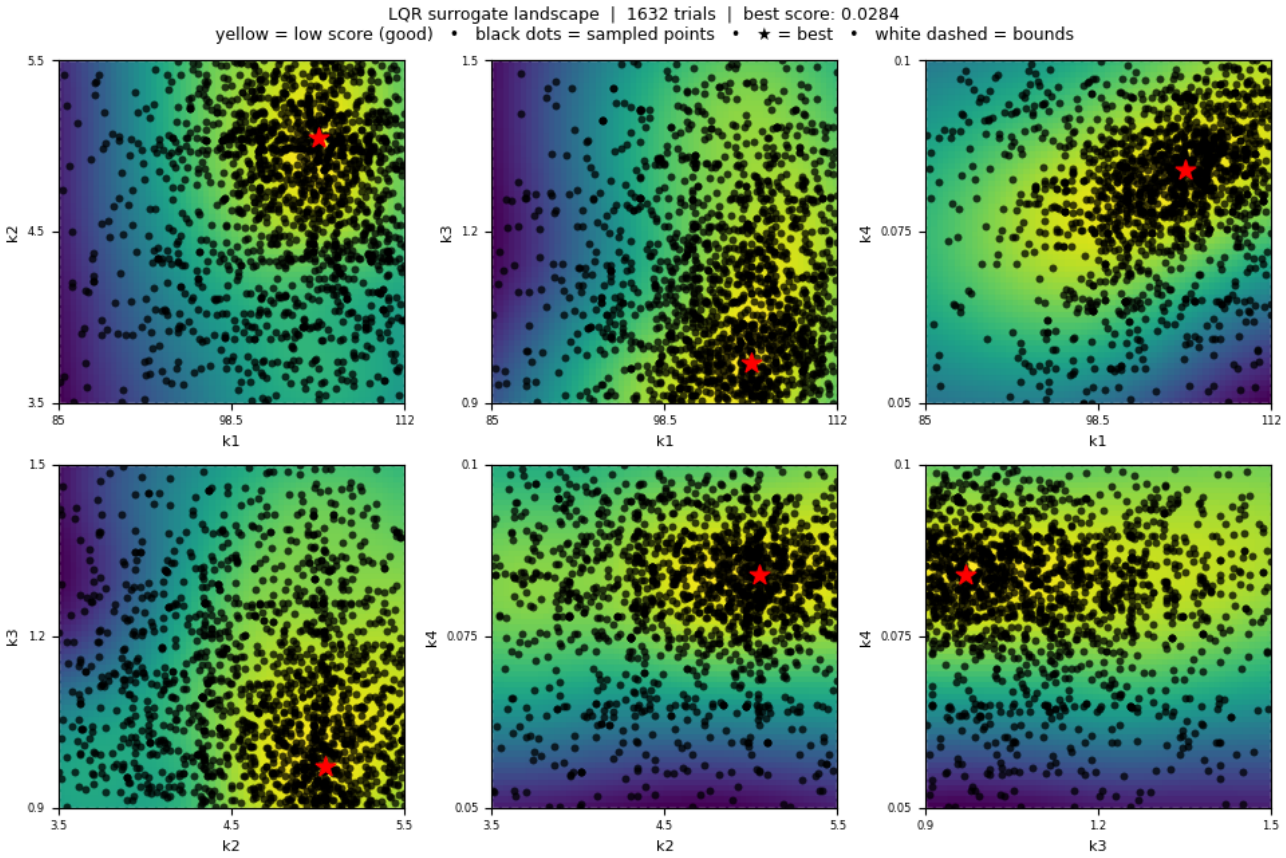


Figure A.5.: LQR Run 5 Phase 2 Heatmap

B PID Heatmaps

B.1 PID Run 1 Phase 1

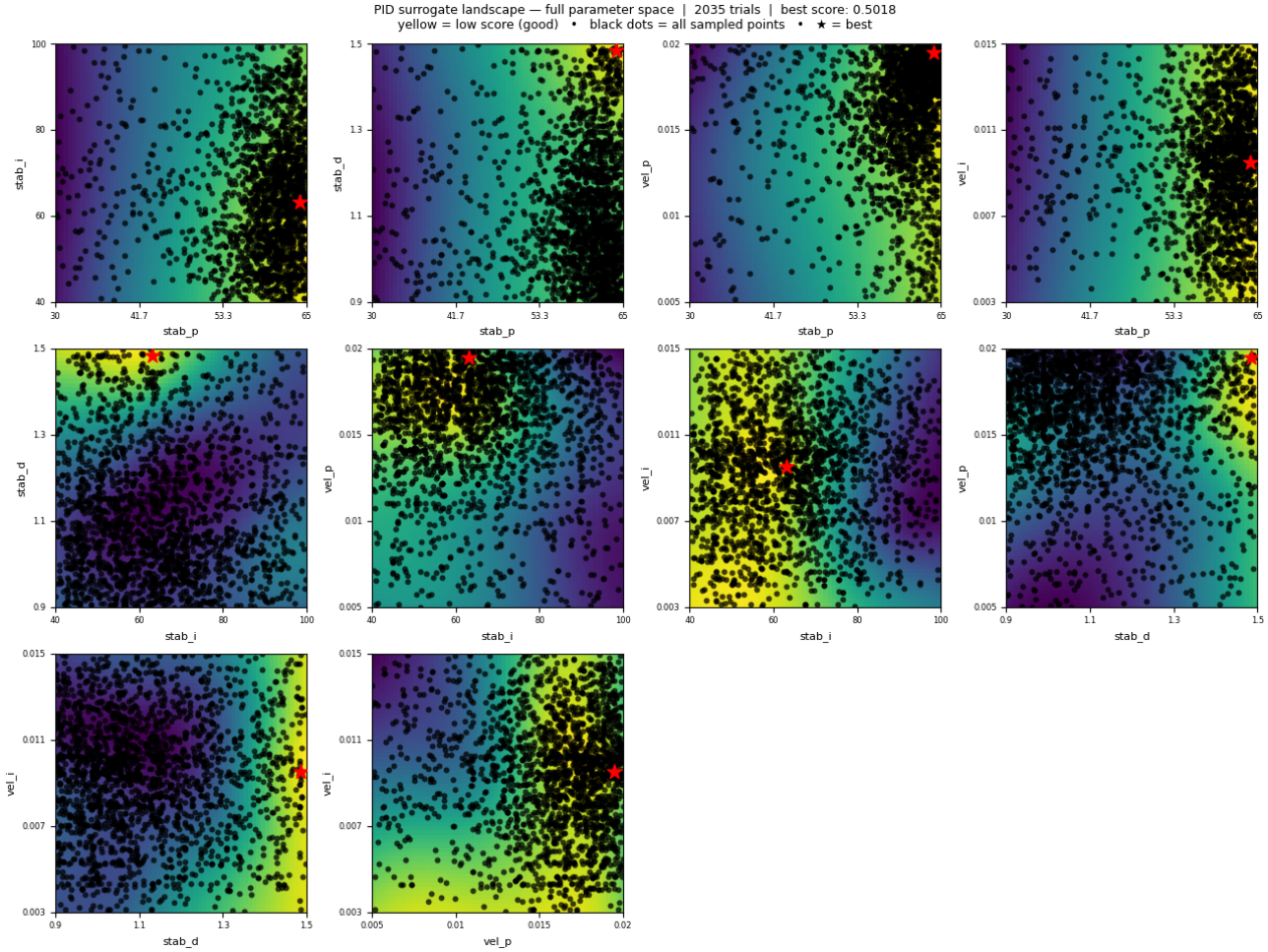


Figure B.1.: PID Run 1 Phase 1 Heatmap

B.2 PID Run 2 Phase 1

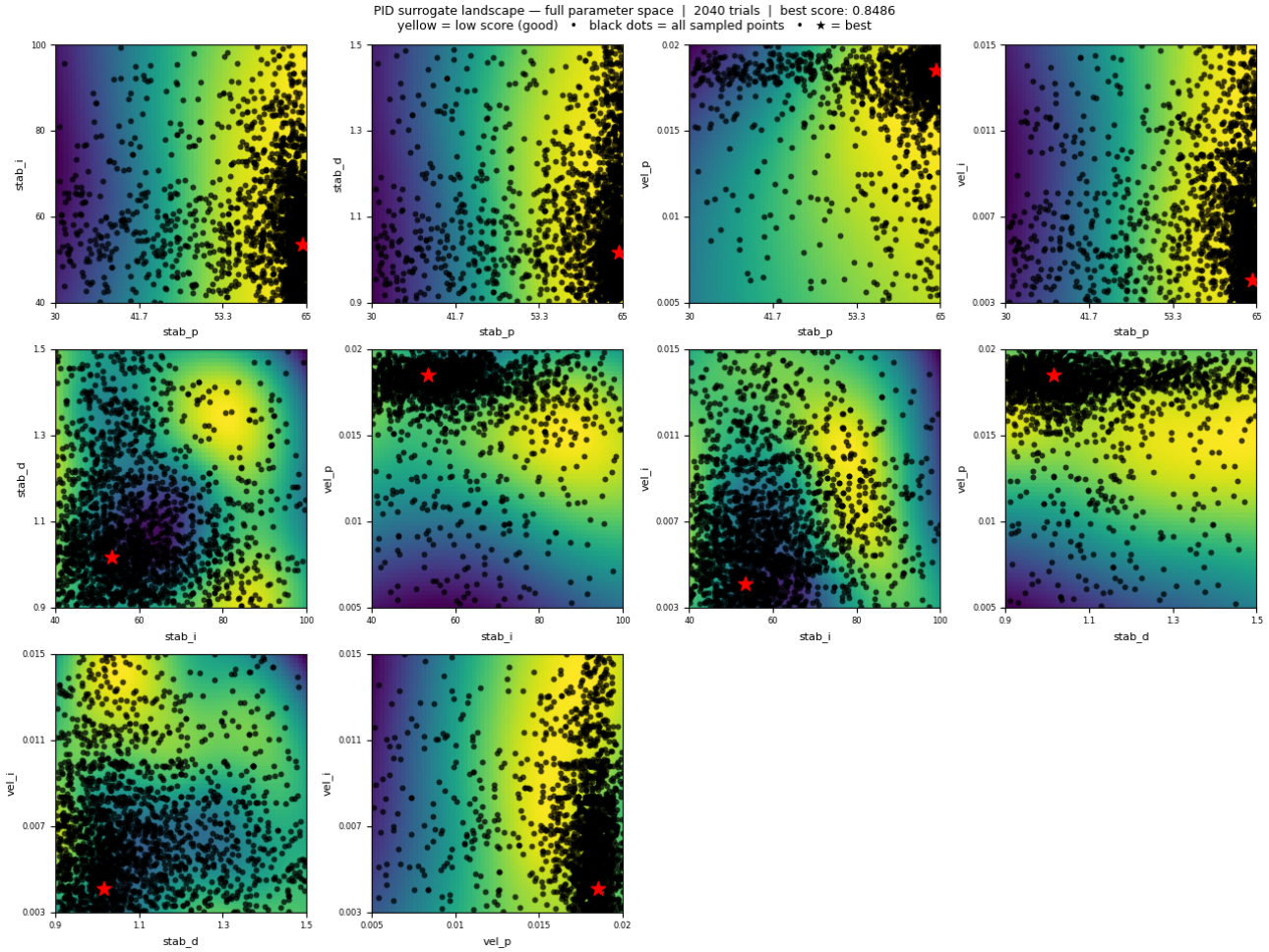


Figure B.2.: PID Run 2 Phase 1 Heatmap

B.3 PID Run 3 Phase 1

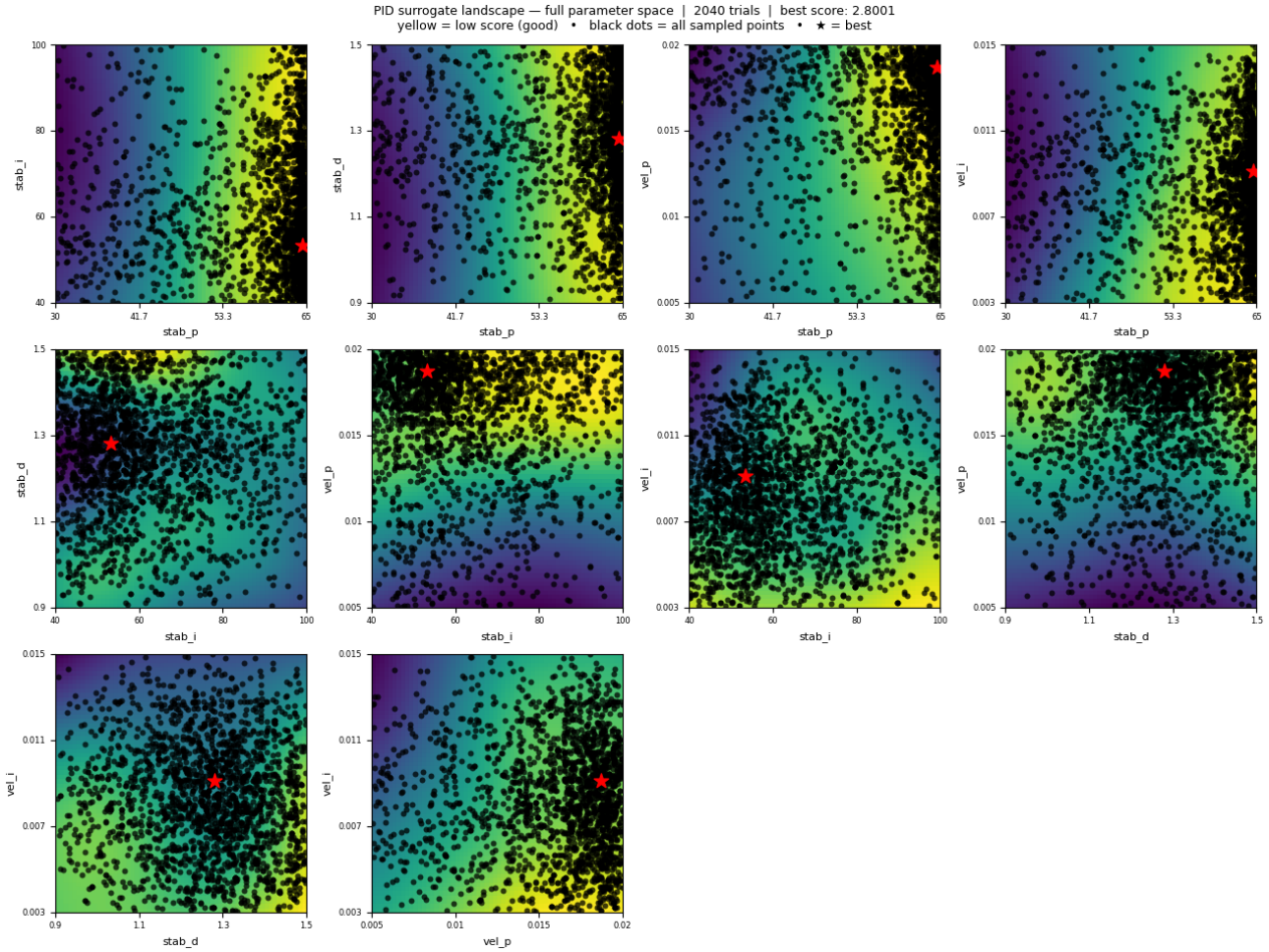


Figure B.3.: PID Run 3 Phase 1 Heatmap

B.4 PID Run 4 Phase 1

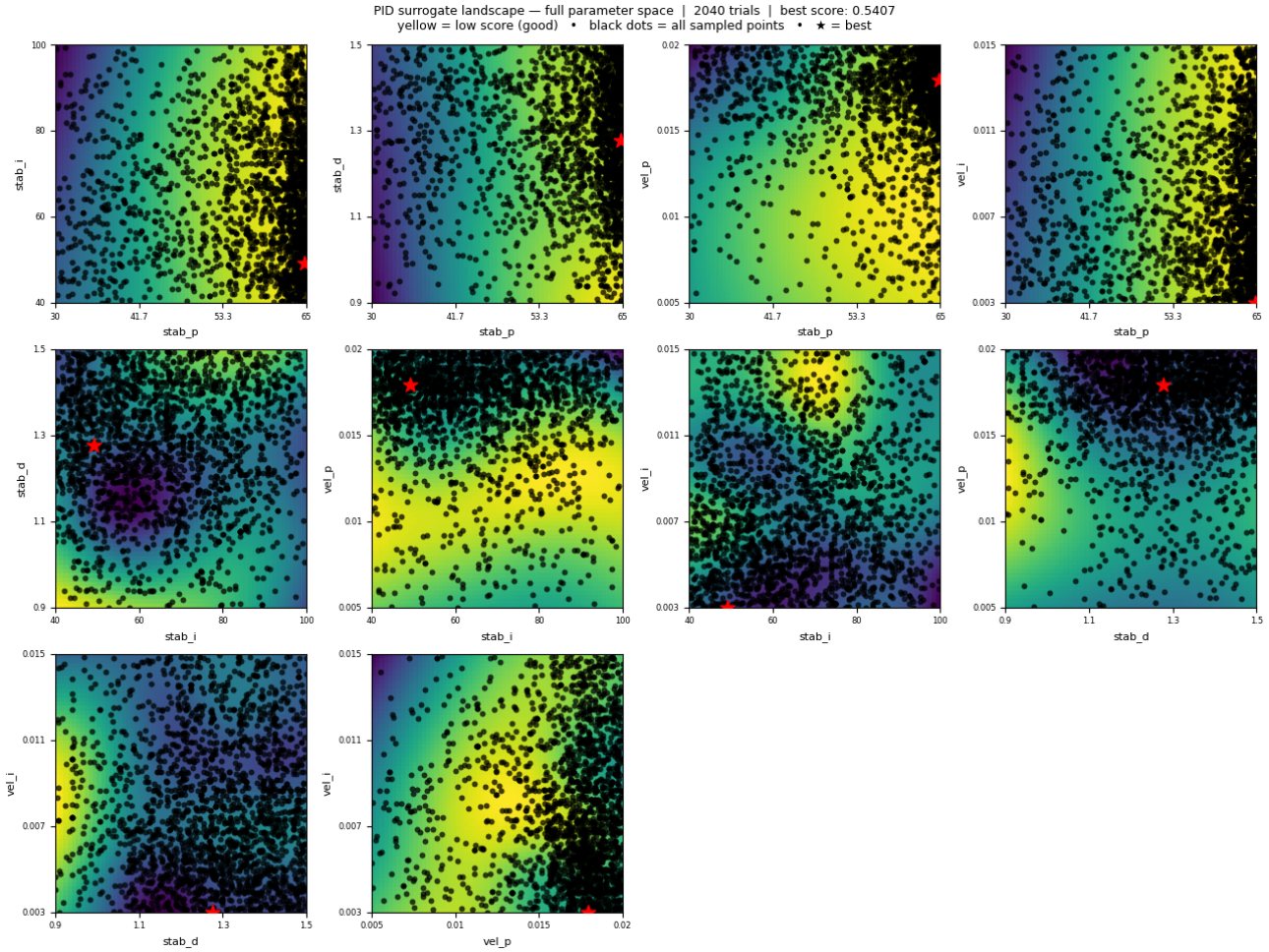


Figure B.4.: PID Run 4 Phase 1 Heatmap

B.5 PID Run 5 Phase 2

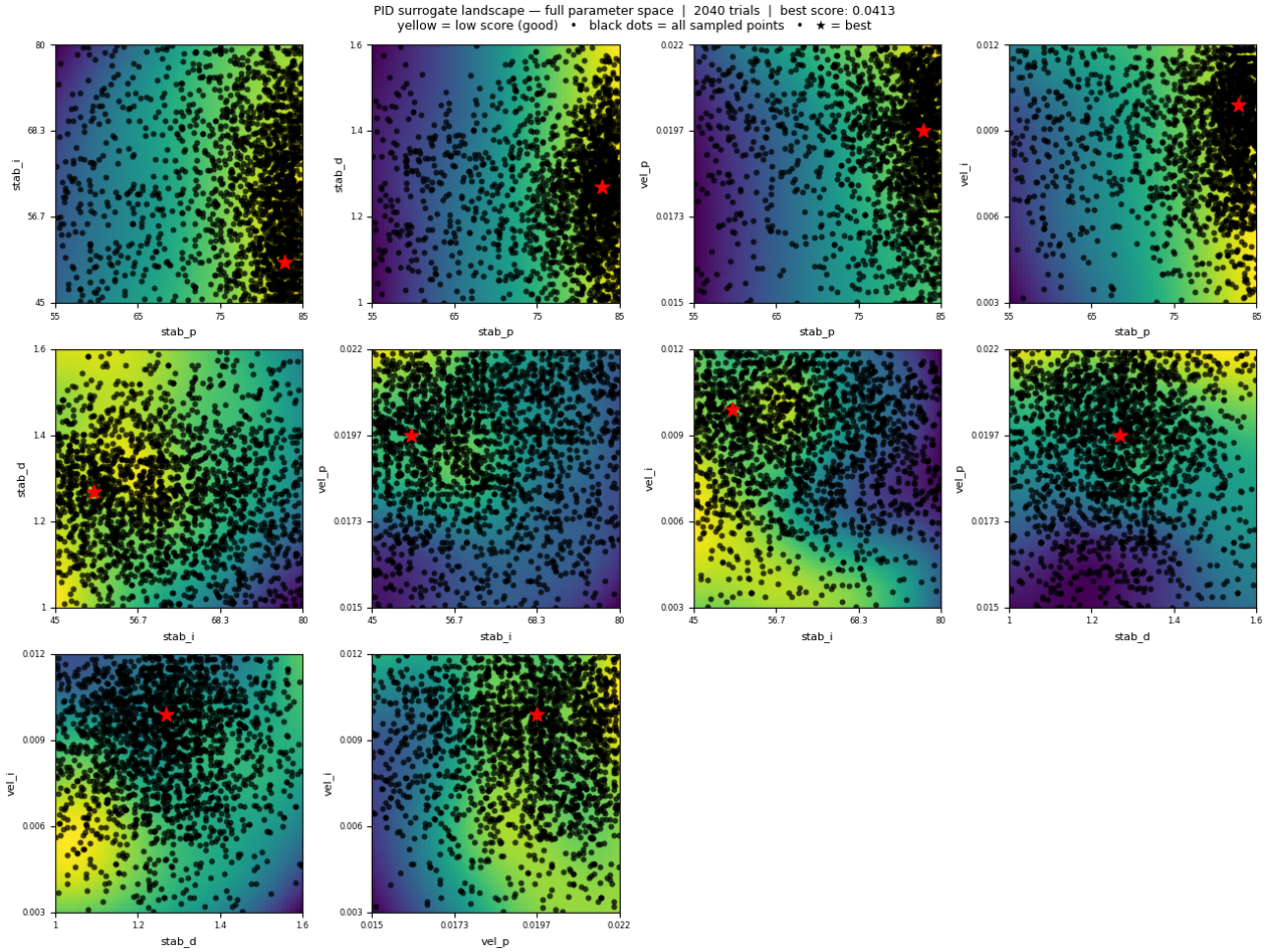


Figure B.5.: PID Run 5 Phase 2 Heatmap

C System Identification

C.1 Free-Fall Test Valid Results

The following table shows the pitch angle recorded during the free-fall sessions, fitted to $\theta(t) = A\cosh(\lambda t) + B\sinh(\lambda t)$.

Run	λ (rad^2/s^2)	$\sigma(\lambda)$	Weight ($1/\sigma^2$)	Notes
3	57.28	0.271	13.6	Valid
4	68.19	0.100	100.0	Best — dominates the mean weight
5	73.47	0.253	15.6	Valid
Weighed mean	67.5	—	—	**Final Value**

Runs from both sessions with initial pitch rate exceeding $15°/s$ were rejected as they indicate the robot was pushed rather than falling freely.

C.2 Free-Fall Fit

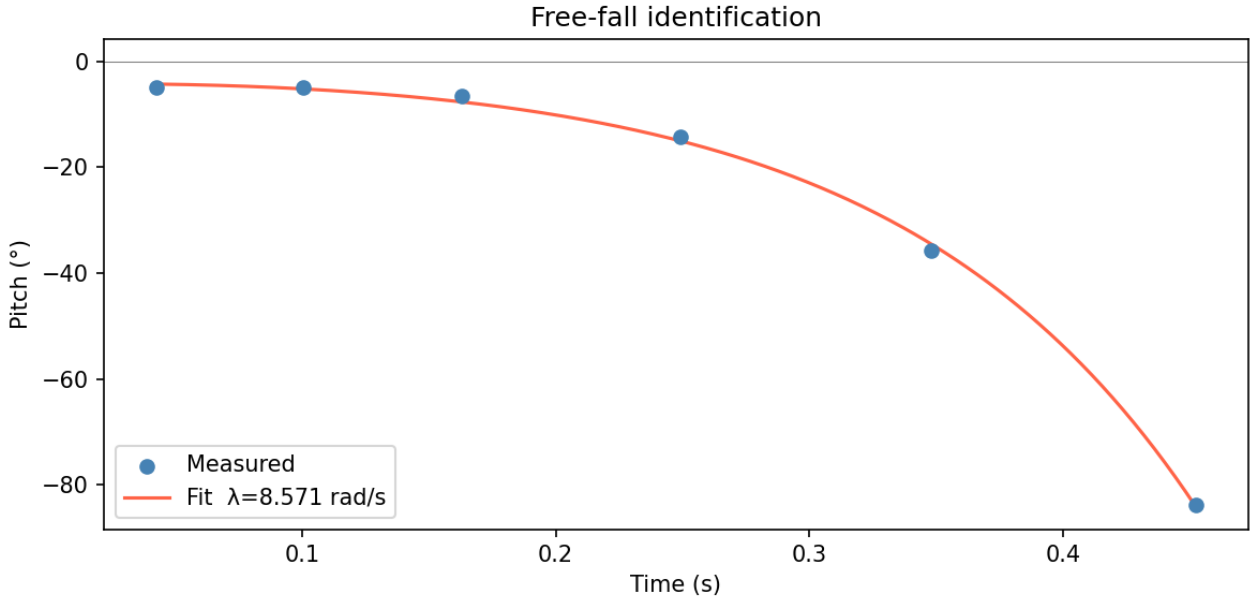


Figure C.1.: Free-fall fit

C.3 B_v Open Loop Pulse Test Results

Run	v_{start} (rad/s)	B_v (rad/s^2/V)	Valid	Notes
1	0.31	0.595	Yes	Clean start
2	0.25	0.541	Yes	Clean start
3	1.78	−0.1234	No	High initial velocity
4	−0.42	0.837	Yes	Backward start, crosses zero
5	1.52	−0.0891	No	High initial velocity